

EcoAlerta

Aplicacion colaborativa para propiciar el cuidado del
medio ambiente

Trabajo Fin de Grado

Grado en Ingenieria Informatica
Universidad de Alicante

Autor: Erardo Aldana Pessoa

Tutor: Jose Luis Sanchez Romero

Convocatoria: C3 – Curso 2025–2026

[BORRADOR – Portada provisional. La portada oficial EPS se incorpora en la version final.]

Resumen

EcoAlerta es una aplicación web progresiva (PWA) concebida como red social cívica especializada en medio ambiente. El sistema permite a cualquier ciudadano reportar incidencias medioambientales (vertidos ilegales, contaminación, incendios forestales, animales abandonados o maltratados, daños a espacios naturales, entre otras categorías) con fotografía y ubicación GPS, clasifica automáticamente el caso por categoría y severidad, y lo delega a la entidad responsable correspondiente (ayuntamientos, SEPRONA, bomberos, policía local) mediante un flujo de estados trazable. Los ciudadanos pueden consultar el mapa público de incidencias sin registrarse y, una vez autenticados, votar, comentar y seguir los reportes de otros usuarios, introduciendo así un componente social que amplifica las incidencias más relevantes.

El proyecto se ha desarrollado siguiendo una metodología iterativa e incremental de seis *sprints* de una semana, aplicando el enfoque *Spec-Driven Development* (SpecKit) para combinar especificaciones formales con asistencia de agentes de inteligencia artificial en el IDE Google Antigravity. El uso de IA queda declarado de manera explícita y transparente en la memoria, reservando al autor todas las decisiones de diseño, la revisión de código y la responsabilidad final del producto.

Tecnológicamente, el sistema se estructura en tres capas desplegadas con Docker Compose: una SPA en React 18 con Tailwind CSS y Leaflet sobre OpenStreetMap como capa de presentación (instalable como PWA con soporte *offline*), una API REST en Node.js 20 y Express 4 como capa de lógica de negocio (con autenticación JWT de doble token, 2FA TOTP, *rate limiting* y Cloudflare Turnstile), y PostgreSQL 16 con PostGIS 3.4 como capa de datos, con un índice GIST que permite consultas geoespaciales eficientes mediante `ST_DWithin`. El modelo de datos consta de trece entidades que soportan incidencias, fotografías, comentarios, votos, seguimientos, historial de estados, notificaciones, y un *audit log* de seguridad completo.

Se han definido 46 requisitos funcionales y 12 no funcionales, organizados en cinco módulos: autenticación, incidencias, mapa interactivo, administración y delegación, y módulo social con notificaciones. La API expone 35 *endpoints* REST documentados con Swagger/OpenAPI. La validación se ha realizado mediante una pirámide de pruebas con Jest (pruebas unitarias y de integración del *backend*), Su-

peritest (pruebas de API) y Playwright (pruebas *end-to-end*) desplegadas automáticamente en cada *commit* mediante GitHub Actions.

Los resultados confirman la viabilidad técnica del sistema: despliegue reproducible con un único comando, cobertura de pruebas superior al 60 % en el *backend*, búsquedas geoespaciales en un radio de 5 km por debajo de 500 ms sobre bases de datos con miles de incidencias, y validación del cumplimiento de todos los requisitos funcionales de prioridad alta. Como trabajo futuro se plantean la apertura a municipios reales mediante convenios con administraciones, la incorporación de una aplicación nativa complementaria para usuarios avanzados, la integración con sistemas de gestión municipal existentes y un análisis más profundo del modelo de negocio en régimen B2G (*business-to-government*).

Palabras clave: aplicación cívica, medio ambiente, PostGIS, PWA, React, Node.js, JWT, Docker, Spec-Driven Development, UML, OpenStreetMap.

Contents

Resumen	i
1 Introducción	3
1.1 Motivación y contexto	3
1.2 Objetivos del proyecto	4
1.3 Uso de herramientas de inteligencia artificial	5
1.4 Estructura de la memoria	7
2 Estado del arte	9
2.1 Contexto: aplicaciones cívicas de reporte geolocalizado	9
2.2 Aplicaciones analizadas	10
2.2.1 FixMyStreet	10
2.2.2 Línea Verde	10
2.2.3 iNaturalist	11
2.2.4 SeeClickFix	11
2.2.5 Epicollect5	12
2.3 Comparativa funcional	12
2.4 Comparativa técnica: tecnologías evaluadas	13
2.4.1 Plataforma: PWA frente a aplicación nativa	13
2.4.2 Framework de <i>frontend</i> : React frente a Vue y Angular	14
2.4.3 Base de datos: PostgreSQL + PostGIS frente a MongoDB	15
2.4.4 Mapa: Leaflet + OpenStreetMap frente a Google Maps	15
2.4.5 Metodología: SDD frente a Scrum tradicional	16
2.5 Posicionamiento de EcoAlerta	16
3 Análisis y diseño	19
3.1 Metodología de desarrollo	19
3.2 Actores del sistema	20
3.3 Requisitos funcionales	21
3.3.1 Módulo de autenticación	21
3.3.2 Módulo de incidencias	22

3.3.3	Módulo de mapa interactivo	23
3.3.4	Módulo de administración y delegación	23
3.3.5	Módulo social y notificaciones	24
3.4	Requisitos no funcionales	25
3.5	Casos de uso	26
3.6	Arquitectura del sistema	28
3.6.1	Stack tecnológico y justificación	30
3.7	Modelo de datos	31
3.7.1	Descripción de las entidades principales	32
3.7.2	Consideraciones de rendimiento geoespacial	34
3.8	Flujo de estados de una incidencia	34
3.9	Diseño de la API REST	35
3.10	Diseño de la interfaz de usuario	37
4	Implementación	39
4.1	Entorno de desarrollo y herramientas	39
4.2	Estructura del repositorio	40
4.3	Desarrollo por sprints	41
4.3.1	Sprint 1 — Autenticación y base de datos	41
4.3.2	Sprint 2 — CRUD de incidencias, fotos y GPS	42
4.3.3	Sprint 3 — Mapa interactivo y filtros geoespaciales	43
4.3.4	Sprint 4 — Panel de administración, delegación y notificaciones	44
4.3.5	Sprint 5 — Testing, PWA y <i>polish</i>	45
4.3.6	Sprint 6 — <i>Security hardening</i>	46
4.4	Aspectos destacados del <i>frontend</i>	47
4.4.1	Navegación y <i>layout</i>	47
4.4.2	Contexto de autenticación	47
4.4.3	Componentes reutilizables clave	47
4.5	Despliegue y orquestación con Docker	47
4.6	Integración continua	48
4.7	Capturas de pantalla de la aplicación	49
4.8	Principales retos técnicos resueltos	49
4.9	Uso de asistencia de inteligencia artificial en la implementación	51
5	Evaluación	53
5.1	Pruebas automatizadas del <i>backend</i>	53
5.2	Pruebas <i>end-to-end</i> con Playwright	54
5.3	Validación del cumplimiento de requisitos	55
5.4	Pruebas de usabilidad con usuarios reales	58
5.5	Valoración global	58

6 Conclusiones y trabajo futuro	61
6.1 Logros alcanzados	61
6.2 Limitaciones	62
6.3 Trabajo futuro	63
6.3.1 Líneas de corto plazo (<i>hardening</i> y despliegue)	64
6.3.2 Líneas de medio plazo (funcionalidad)	64
6.3.3 Líneas de largo plazo (producto)	64
6.4 Reflexión personal	65
A Guía de instalación	67
B Manual de usuario	71
B.1 Perfil: ciudadano anónimo	71
B.2 Perfil: ciudadano registrado	72
B.3 Perfil: administrador	73
B.4 Perfil: entidad responsable	74
B.5 Accesibilidad y atajos de teclado	74
C Modelo de negocio preliminar	75
C.1 Propuesta de valor	75
C.2 Segmentos de mercado	75
C.3 Fuentes de ingresos	76
C.4 Estructura de costes	77
C.5 Análisis de viabilidad	77
C.6 Aspectos legales	77
C.7 Conclusión del análisis	78

List of Figures

3.1	Diagrama de casos de uso de EcoAlerta. Fuente: elaboración propia a partir de los requisitos RF-01–RF-46.	27
3.2	Diagrama de arquitectura de componentes de EcoAlerta.	29
3.3	Modelo entidad-relación de EcoAlerta (13 tablas).	32
3.4	Diagrama UML de estados de una incidencia. Cada transición se almacena en <code>incident_status_history</code> con <i>timestamp</i> y usuario responsable.	35
4.1	Flujos del ciudadano registrado. (Capturas pendientes de incorporar por el autor.)	49
4.2	Vistas adicionales: consulta de detalle y gestión administrativa. (Capturas pendientes de incorporar.)	49

List of Tables

2.1	Comparativa funcional de las aplicaciones analizadas y EcoAlerta.	12
2.2	Comparación entre PWA y aplicación nativa.	14
2.3	Comparativa de frameworks de <i>frontend</i>	15
3.1	Actores del sistema EcoAlerta.	21
3.2	Requisitos funcionales del módulo de autenticación.	22
3.3	Requisitos funcionales del módulo de incidencias.	22
3.4	Requisitos funcionales del módulo de mapa interactivo.	23
3.5	Requisitos funcionales del módulo de administración.	24
3.6	Requisitos funcionales del módulo social y de notificaciones.	24
3.7	Requisitos no funcionales del sistema.	25
3.8	Stack tecnológico empleado y justificación de cada elección.	30
3.9	Endpoints principales de la API REST.	36
4.1	Herramientas del entorno de desarrollo.	39
4.2	Resumen de los seis sprints de desarrollo.	41
4.3	Pirámide de pruebas del proyecto.	45
4.4	Cobertura de pruebas del <i>backend</i> (reporte Jest).	45
5.1	Ficheros de prueba del <i>backend</i>	54
5.2	Cobertura final del <i>backend</i>	54
5.3	Pruebas E2E con Playwright.	55
5.4	Validación del cumplimiento de requisitos funcionales.	55
5.5	Validación del cumplimiento de requisitos no funcionales.	57
5.6	Resultados preliminares de la prueba de usabilidad (plantilla).	58
C.1	Planes de suscripción propuestos.	76

Chapter 1

Introducción

1.1 Motivación y contexto

La preocupación por la conservación del medio ambiente es uno de los ejes transversales de la agenda pública europea y nacional. España enfrenta en la presente década un conjunto de retos medioambientales particularmente severos: incendios forestales recurrentes asociados al cambio climático, contaminación atmosférica en entornos urbanos, vertidos ilegales en suelos rústicos y espacios naturales, y maltrato y abandono de animales, entre otros. La Ley 7/2022 de residuos y suelos contaminados, la reciente Ley de bienestar animal y la adhesión al Pacto Verde Europeo enmarcan una política pública exigente, pero su efectividad depende en gran medida de la capacidad de las administraciones para **detectar a tiempo** los problemas que ocurren sobre el terreno.

A la capacidad institucional de detección se suma una ventaja infrautilizada: la ciudadanía dispone hoy de dispositivos móviles con cámara y GPS, conectividad casi permanente y una creciente voluntad de participación. El Instituto Nacional de Estadística estimó en 2023 que el 97,7 % de los hogares españoles cuenta con acceso a Internet y más del 95 % dispone de un *smartphone* con los sensores necesarios para registrar una incidencia medioambiental con prueba fotográfica geolocalizada. Sin embargo, los canales institucionales tradicionales de denuncia (teléfono 112, oficinas presenciales, formularios web genéricos no geolocalizados) no están diseñados para absorber este flujo potencial ni para proporcionar seguimiento al denunciante.

Los datos disponibles confirman la magnitud del problema: el *Boletín de información sobre emergencias y protección civil* del Ministerio del Interior registra más de diez mil incendios forestales anuales en España; el SEPRONA (Servicio de Protección de la Naturaleza de la Guardia Civil) tramita cada año más de 130 000 actuaciones relacionadas con infracciones y delitos medioambientales; y organismos como las confederaciones hidrográficas documentan miles de vertidos a cauces públicos. Sin

embargo, un porcentaje significativo de estas incidencias se detectan por denuncia ciudadana tardía o por inspección reactiva, no por un canal ágil y sistemático. La brecha entre *lo que la ciudadanía ve* y *lo que llega a las administraciones* es, por lo tanto, el espacio problema en el que se inserta el presente Trabajo Fin de Grado.

En el plano internacional, existen aplicaciones que han demostrado la viabilidad de cubrir esta brecha: *FixMyStreet* (Reino Unido) acumula desde 2007 más de dos millones de incidencias reportadas por ciudadanos que han sido comunicadas a autoridades locales; *iNaturalist* agrega más de 180 millones de observaciones de biodiversidad aportadas por voluntarios. En España, *Línea Verde* ofrece un canal municipal de quejas al ayuntamiento en más de trescientas localidades. Estas referencias, analizadas en detalle en el Capítulo 2, demuestran que el enfoque es técnicamente factible y socialmente deseable, pero también evidencian huecos importantes: ninguna de las aplicaciones disponibles combina a la vez reporte geolocalizado con foto, delegación automatizada a la entidad responsable (ayuntamiento, SEPRONA, bomberos), niveles de severidad adaptados al impacto ambiental y un componente social que amplifique las incidencias más relevantes mediante votos y seguidores.

El presente Trabajo Fin de Grado propone **EcoAlerta**, una aplicación web progresiva (PWA) concebida como red social cívica especializada en medio ambiente. La aplicación permite a cualquier ciudadano reportar una incidencia con foto y GPS en menos de un minuto, clasifica automáticamente el caso por categoría y severidad, lo delega a la entidad responsable correspondiente mediante un flujo de estados trazable, y permite a otros ciudadanos votar y seguir la evolución del caso. El sistema está diseñado para ofrecer un equilibrio entre valor público (cualquiera puede consultar el mapa de incidencias como herramienta de transparencia) y responsabilidad (las acciones con impacto requieren autenticación y quedan registradas).

1.2 Objetivos del proyecto

El objetivo general del proyecto puede enunciarse como sigue:

Diseñar, implementar y validar una aplicación web progresiva, denominada EcoAlerta, que facilite a la ciudadanía el reporte geolocalizado de incidencias medioambientales y permita su delegación automatizada a las entidades responsables, con un flujo de estados trazable y una componente social de apoyo comunitario.

Este objetivo general se desglosa en ocho objetivos específicos:

- OBJ-1** Analizar el estado del arte de aplicaciones cívicas y de participación medioambiental, identificando requisitos funcionales relevantes y huecos de cobertura en las soluciones existentes.

- OBJ-2** Definir una arquitectura escalable basada en contenedores, con separación estricta entre capa de presentación, capa de lógica de negocio y capa de datos, y con soporte nativo para consultas geoespaciales.
- OBJ-3** Diseñar un modelo de datos relacional con extensión geoespacial (Post-GIS) capaz de representar incidencias, fotos, actores, flujos de estado y eventos sociales con un *audit trail* completo.
- OBJ-4** Implementar una API REST documentada (OpenAPI/Swagger) que exponga la funcionalidad del sistema con control de acceso basado en roles y *rate limiting*.
- OBJ-5** Desarrollar una interfaz web progresiva *mobile-first*, instalable, con soporte *offline* básico, mapa interactivo, y que cumpla las recomendaciones de accesibilidad WCAG 2.1 AA.
- OBJ-6** Incorporar mecanismos de seguridad propios de aplicaciones en producción: autenticación JWT con rotación de tokens, autenticación de doble factor, *captcha* invisible, *audit log* estructurado y cifrado de secretos sensibles.
- OBJ-7** Validar el sistema mediante una pirámide de pruebas (unitarias, de integración, *end-to-end*) y pruebas de usabilidad con usuarios reales, verificando el cumplimiento de los requisitos funcionales y no funcionales.
- OBJ-8** Producir documentación técnica reproducible (despliegue con un único comando, documentación API autogenerada, manual de usuario) y un estudio preliminar de modelo de negocio que oriente una eventual puesta en producción.

1.3 Uso de herramientas de inteligencia artificial

La elaboración de este Trabajo Fin de Grado ha hecho uso, de manera consciente y declarada, de herramientas de inteligencia artificial generativa en tareas concretas, siguiendo las recomendaciones de la Universidad de Alicante sobre integridad académica en contextos donde la asistencia de IA está permitida siempre que sea explícitamente reconocida. Esta sección describe con precisión qué uso se ha hecho, para qué tareas, qué responsabilidades han quedado en manos del autor, y qué mecanismos se han puesto en marcha para garantizar que el resultado final sea fruto de un proceso de autoría real.

Herramientas utilizadas. Se han empleado agentes de IA basados en los modelos de lenguaje *Claude* (Anthropic) y *Gemini* (Google), este último integrado en el IDE *Google Antigravity* bajo un paradigma de desarrollo por agentes.

Tareas asistidas. La IA ha asistido en las siguientes tareas específicas:

- Generación inicial de *boilerplate* de código (estructura de rutas Express, componentes React, configuraciones de Docker Compose).
- Refactorización y optimización de fragmentos de código previamente escritos por el autor.
- Generación de pruebas Jest a partir de los contratos de los servicios.
- Redacción de borradores de la memoria a partir de la especificación del proyecto y las notas del autor.
- Revisión ortográfica y de estilo académico del texto final.

Tareas realizadas exclusivamente por el autor. Las siguientes tareas no han delegado en la IA:

- Definición de la idea, el alcance y los objetivos del proyecto.
- Toma de decisiones arquitectónicas (elección de *stack*, modelo de datos, flujo de estados).
- Análisis de las aplicaciones de referencia del Capítulo 2: descarga, uso efectivo y redacción de observaciones propias.
- Pruebas de usabilidad con usuarios reales y análisis de resultados.
- Revisión crítica de todo el código generado y validación mediante ejecución local.
- Redacción y argumentación de los capítulos de análisis (Capítulo 3), evaluación (Capítulo 5) y conclusiones (Capítulo 6).

Mecanismos de garantía de autoría. El proyecto se ha desarrollado sobre un repositorio Git público donde cada *commit* refleja un paso del trabajo, siguiendo la convención *Conventional Commits*. El historial es auditable y permite reconstruir el proceso de construcción. Todas las especificaciones de *sprint* se escribieron por el autor antes de solicitar asistencia a los agentes, y están disponibles en `docs/sprints/` del repositorio. Para la defensa oral, el autor asume plena responsabilidad de explicar y justificar cualquier fragmento de código, decisión de diseño o contenido de la memoria, con independencia de si el texto o el código fue generado inicialmente con asistencia.

Posicionamiento del autor. La aportación de valor del presente TFG reside, a juicio del autor, en tres elementos no sustituibles por IA: la identificación del hueco de cobertura que aborda EcoAlerta frente a aplicaciones existentes, el diseño integrado del flujo ciudadano–administrador–entidad–sistema, y la ingeniería de requisitos que ha conducido de una idea inicial a un sistema con 46 requisitos funcionales verificables. La IA ha sido, en palabras simples, una herramienta de productividad; las decisiones y la responsabilidad pertenecen al autor.

1.4 Estructura de la memoria

El resto de la memoria se organiza en cinco capítulos y tres anexos:

Capítulo 2, Estado del arte.

Análisis de cinco aplicaciones de referencia (FixMyStreet, Línea Verde, iNaturalist, SeeClickFix, Epicollect5) y comparativa técnica de las tecnologías evaluadas.

Capítulo 3, Análisis y diseño.

Requisitos funcionales y no funcionales, actores, casos de uso, arquitectura, modelo de datos y diseño de API.

Capítulo 4, Implementación.

Desarrollo del sistema organizado en seis *sprints*, con decisiones técnicas y extractos de código significativos.

Capítulo 5, Evaluación.

Resultados de la pirámide de pruebas, validación del cumplimiento de requisitos, pruebas de usabilidad con usuarios reales.

Capítulo 6, Conclusiones.

Síntesis de logros, limitaciones honestas del trabajo y líneas de trabajo futuro.

Anexo A, Guía de instalación.

Requisitos previos, instrucciones paso a paso para levantar el entorno y solución de problemas frecuentes.

Anexo B, Manual de usuario.

Guía visual de cada perfil (ciudadano, administrador, entidad) con capturas de pantalla.

Anexo C, Modelo de negocio.

Análisis de sostenibilidad económica del proyecto en una hipotética puesta en producción.

Los *listings* de figuras, tablas, acrónimos y la bibliografía completa se incluyen al final del documento.

Chapter 2

Estado del arte

Este capítulo revisa las aplicaciones de referencia en el ámbito de la participación ciudadana con vocación medioambiental o cívica, analiza las tecnologías candidatas empleadas en cada una, y justifica las decisiones técnicas adoptadas en EcoAlerta sobre la base de esa revisión.

2.1 Contexto: aplicaciones cívicas de reporte geolocalizado

Las aplicaciones cívicas de reporte geolocalizado se enmarcan en el movimiento *civic tech*, que desde mediados de los años 2000 ha promovido el uso de tecnologías abiertas para facilitar la participación ciudadana en asuntos públicos. Tres innovaciones han sido especialmente catalizadoras:

1. La llegada del *smartphone* con cámara y GPS (iPhone 2007, Android 2008) hizo viable que cualquier ciudadano pudiera generar evidencia fotográfica geolocalizada en tiempo real.
2. Los mapas libres *OpenStreetMap* (2004) democratizaron la cartografía y eliminaron la dependencia de proveedores cerrados.
3. El movimiento de datos abiertos (*open data*) y leyes de transparencia (ley española 19/2013) obligaron a las administraciones a publicar información y crearon un marco para el intercambio bidireccional ciudadano–administración.

Desde entonces, se han consolidado dos familias principales de aplicaciones cívicas: las **generalistas** (como *FixMyStreet* o *SeeClickFix*, que cubren cualquier incidencia urbana) y las **especializadas** (como *iNaturalist* para observación de biodiversidad, o *Línea Verde* en España para reporte a ayuntamientos). EcoAlerta se posiciona como una aplicación especializada en medio ambiente pero con componente social (votos, comentarios, seguimiento), cubriendo un hueco entre ambas familias.

2.2 Aplicaciones analizadas

Se han seleccionado cinco aplicaciones de referencia por su relevancia, madurez y heterogeneidad tecnológica: FixMyStreet, Línea Verde, iNaturalist, SeeClickFix y Epicollect5. Las tres primeras se analizan con mayor profundidad al haber sido **probadas directamente** por el autor en un dispositivo móvil siguiendo la plantilla de análisis docs/estado-del-arte/plantilla-analisis.docx. Las dos últimas se analizan documentalmente.

Para cada aplicación se recoge: origen y trayectoria, modelo de datos conceptual visible desde la interfaz, funcionalidades disponibles, flujo de trabajo principal, tecnologías empleadas (cuando se han podido identificar), y observaciones personales sobre la experiencia de uso y oportunidades de mejora.

2.2.1 FixMyStreet

Origen. FixMyStreet fue desarrollado en 2007 por mySociety, una organización británica sin ánimo de lucro dedicada a *civic tech*. Su código es abierto (AGPL-3.0) y ha sido reutilizado por cientos de ayuntamientos en todo el mundo, incluyendo la versión *FixMyStreet Bruxelles* analizada en este trabajo.

Funcionalidades observadas.

Flujo de trabajo.

Tecnología. Aplicación web con diseño *mobile-friendly*. Emplea mapas OpenStreetMap. El *backend* es Perl con Plack y PostgreSQL.

Puntos fuertes.

Puntos débiles / oportunidades de mejora.

2.2.2 Línea Verde

Origen. Línea Verde es un servicio comercial de la empresa *Línea Verde Configuración y Servicios Medioambientales S.A.* implantado en más de 300 municipios españoles. Su enfoque es B2G: cada ayuntamiento contrata el servicio y obtiene una versión blanqueada (*white-label*) para sus ciudadanos.

Funcionalidades observadas.

Flujo de trabajo.

Tecnología. Aplicación móvil nativa (iOS/Android) y portal web. El *backend* no es público.

Puntos fuertes.

Puntos débiles / oportunidades de mejora.

2.2.3 iNaturalist

Origen. Proyecto nacido en 2008 en la Universidad de California como plataforma de ciencia ciudadana para observación de biodiversidad. Gestionado conjuntamente por la California Academy of Sciences y la National Geographic Society. Acumula más de 180 millones de observaciones a escala mundial.

Funcionalidades observadas.

Flujo de trabajo.

Tecnología. Aplicación móvil nativa y web. Emplea Ruby on Rails en el *backend*, PostgreSQL con PostGIS, Elasticsearch para búsqueda y un servicio de reconocimiento de imagen basado en redes neuronales para sugerir la especie de la observación.

Puntos fuertes.

Puntos débiles / oportunidades de mejora.

2.2.4 SeeClickFix

Origen. Fundado en 2008 en New Haven (Connecticut, EE.UU.), se convirtió en uno de los referentes estadounidenses del reporte cívico. En 2019 fue adquirido por CivicPlus y continúa operando en numerosos municipios de Norteamérica.

Funcionalidades destacadas. Reporte geolocalizado con foto, categorías estandarizadas, flujo de estados (recibido, reconocido, en curso, resuelto), panel administrativo para el municipio, e integración con sistemas *backend* de gestión municipal americanos mediante API.

Tecnología. Aplicación móvil nativa (iOS/Android) y web. *Backend* no público. API documentada para integraciones.

Relevancia para EcoAlerta. SeeClickFix valida la viabilidad del modelo B2G con flujo de estados trazable. Su principal limitación desde la perspectiva de EcoAlerta es el foco urbano exclusivo: no aborda explícitamente incendios forestales, fauna silvestre o daños a espacios naturales protegidos.

2.2.5 Epicollect5

Origen. Proyecto del *Imperial College London* para la recolección de datos de campo en investigación científica. Especialmente usado en estudios epidemiológicos, conservación y ecología.

Funcionalidades destacadas. Creación de formularios personalizados, recolección *offline* con sincronización diferida, exportación estructurada en CSV/JSON. Totalmente gratuito para investigación.

Tecnología. Aplicación móvil y web. No documenta su *stack*.

Relevancia para EcoAlerta. Epicollect5 es relevante por dos motivos: su robustez en la recolección *offline-first* (que EcoAlerta ha tomado como referencia para el RF-19) y su enfoque generalista, que lo hace una herramienta de propósito demasiado amplio para el ciudadano medio pero muy útil para ONG conservacionistas que pueden configurar formularios específicos.

2.3 Comparativa funcional

La Tabla 2.1 resume las funcionalidades comparadas de las cinco aplicaciones. Se utiliza  para funcionalidad plenamente disponible,  para funcionalidad parcial o con restricciones, y  para ausencia.

Table 2.1: Comparativa funcional de las aplicaciones analizadas y EcoAlerta.

Funcionalidad	FixMy-Street	Línea Verde	iNaturalist	SeeClick-Fix	Epi-collect	Eco-Alerta
Reporte geolocalizado con foto	✓	✓	✓	✓	✓	✓
Enfoque medioambiental específico	×	~	✓	×	~	✓

Funcionalidad	FixMy-Street	Línea Verde	iNaturalist	SeeClick-Fix	Epi-collect	Eco-Alerta
Delegación automatizada a entidades	~	✓	×	✓	×	✓
Flujo de estados trazable	✓	~	×	✓	×	✓
Componente social (votos, comentarios)	~	×	✓	~	×	✓
Perfil público de usuario	×	×	✓	~	×	✓
Acceso anónimo de consulta	✓	✓	✓	✓	~	✓
Niveles de severidad explícitos	~	×	×	~	✓	✓
<i>Audit trail</i> completo	×	×	×	~	×	✓
PWA instalable	~	×	×	×	~	✓
Soporte <i>offline</i>	×	×	✓	×	✓	✓
2FA / <i>audit</i> seguridad	×	×	×	×	×	✓
<i>Open source</i>	✓	×	✓	×	~	✓

La tabla revela que **ninguna de las aplicaciones existentes cubre simultáneamente** las seis dimensiones que EcoAlerta busca atender: enfoque medioambiental específico, delegación automatizada a entidades responsables heterogéneas (no sólo municipales), flujo de estados trazable, componente social significativo, niveles de severidad con impacto en la priorización, y *audit trail* completo. Este es el hueco de cobertura identificado (OBJ-1) y el punto de partida de la propuesta.

2.4 Comparativa técnica: tecnologías evaluadas

Más allá del análisis funcional, se ha realizado una comparativa técnica de las principales opciones disponibles para cada capa. En esta sección se discuten las alternativas consideradas y se justifica cada elección. Las decisiones finales se recogen en la sección 3.6.1.

2.4.1 Plataforma: PWA frente a aplicación nativa

Las aplicaciones cívicas se enfrentan a un dilema clásico: desarrollar una aplicación nativa por plataforma (iOS/Android) o una aplicación web progresiva. Las ventajas

de cada opción se resumen en la Tabla 2.2.

Table 2.2: Comparación entre PWA y aplicación nativa.

Dimensión	PWA	Nativa
Coste de desarrollo	Menor (una <i>code base</i>)	Mayor (dos <i>code bases</i>)
Tiempo de <i>time-to-market</i>	Menor	Mayor
Ciclo de <i>update</i>	Instantáneo (el navegador carga la última versión)	Depende de la <i>app store</i>
Rendimiento	Ligeramente inferior	Nativo
Acceso a hardware	Limitado: cámara, GPS, sensores básicos	Completo
Notificaciones <i>push</i>	Sí, con restricciones (Safari iOS tardó en soportarlo)	Completo
Distribución	Vía URL, no requiere tienda	Requiere Google Play / App Store
Coste de mantenimiento	Menor	Mayor

Para un proyecto con las restricciones temporales de un TFG (aproximadamente 12 créditos ECTS, equivalentes a 300 horas) y con un público objetivo heterogéneo en dispositivos, la PWA es la elección razonable. Se obtiene una *code base* única, se distribuye por URL (facilitando la demostración al tutor y al tribunal) y se cubre el 95 % del caso de uso con un 20 % del esfuerzo que exigiría una aplicación nativa.

La aplicación nativa complementaria se ha propuesto como línea de trabajo futuro (sección 6.3) precisamente para no renunciar a las ventajas que una aplicación nativa aporta (por ejemplo, notificaciones *push* completas y acceso a sensores avanzados).

2.4.2 Framework de *frontend*: React frente a Vue y Angular

Los tres frameworks mayoritarios son candidatos válidos. La Tabla 2.3 resume las consideraciones principales.

Table 2.3: Comparativa de frameworks de *frontend*.

Criterio	React	Vue	Angular
Curva de aprendizaje	Media (JSX)	Suave	Pronunciada (TS + RxJS)
Ecosistema	Muy amplio	Amplio	Amplio pero opinado
Integración con Leaflet	Excelente (<code>react-leaflet</code>)	Buena	Buena
Tamaño del bundle	Pequeño	Muy pequeño	Grande
Mercado laboral en España	Alto	Medio	Alto

Se ha elegido React por tres motivos: es el framework con mayor implantación en el mercado laboral, su integración con Leaflet mediante `react-leaflet` es la más madura de las tres opciones, y la familiaridad del autor con React lograda en asignaturas previas del Grado reduce el tiempo de aprendizaje respecto a una alternativa nueva.

2.4.3 Base de datos: PostgreSQL + PostGIS frente a MongoDB

La elección entre base relacional y NoSQL orientada a documentos es crítica para una aplicación con componente geoespacial. PostgreSQL con PostGIS ofrece tipos geométricos nativos, funciones espaciales (`ST_DWithin`, `ST_Intersects`) y un índice GIST de muy alto rendimiento. MongoDB soporta índices geoespaciales (`2dsphere`) desde hace años, pero carece de varias funciones espaciales avanzadas y de un sistema de transacciones tan robusto para operaciones complejas como las transiciones de estado con *audit log*.

La decisión a favor de PostgreSQL + PostGIS se refuerza al considerar la naturaleza altamente relacional de los datos: las incidencias tienen relaciones claras con fotos, comentarios, votos, usuarios y entidades, todas ellas con restricciones de integridad referencial que el modelo relacional expresa de forma natural. Forzar esto en MongoDB habría requerido *denormalizar* el modelo, con los riesgos de inconsistencia que eso conlleva.

2.4.4 Mapa: Leaflet + OpenStreetMap frente a Google Maps

Google Maps ofrece un rendimiento ligeramente superior y algunas funcionalidades avanzadas (Street View, Places API con reseñas), pero presenta dos inconvenientes

relevantes para el proyecto: (a) coste económico una vez superado el *free tier*, lo que complica el modelo de negocio de la sección C.3, y (b) dependencia de un proveedor comercial con políticas cambiantes. Leaflet + OpenStreetMap es *open source*, gratuito y suficientemente potente para el caso de uso. La decisión a favor de Leaflet se alinea con la filosofía *civic tech* de las aplicaciones de referencia (FixMyStreet y OpenStreetMap comparten raíz común).

2.4.5 Metodología: SDD frente a Scrum tradicional

Se consideraron dos aproximaciones metodológicas: un Scrum clásico por *sprints* de dos semanas con *backlog* y *daily stand-up*, y una aproximación Spec-Driven Development (SDD) con *sprints* de una semana y asistencia de IA. Para un equipo de un único desarrollador, Scrum clásico se vuelve redundante (no hay equipo con el que sincronizarse); en cambio, SDD estructura el trabajo en torno a un artefacto fundamental (la especificación), lo que combina bien con el uso de agentes de IA: una buena especificación permite al agente generar código correcto, y la especificación sirve a la vez como documentación y como guía para la revisión humana. Esta combinación se discute en mayor detalle en el Capítulo 4.

2.5 Posicionamiento de EcoAlerta

A partir del análisis anterior, EcoAlerta se posiciona como una aplicación **especializada en medio ambiente** que combina elementos tomados de varias fuentes:

- El **flujo de estados trazable** de FixMyStreet y SeeClickFix, extendido con transiciones específicas (validación, marcado de duplicados, escalado por *time-out*).
- La **delegación B2G multi-entidad** de Línea Verde, generalizada para soportar también entidades no municipales (SEPRONA, bomberos, ONG).
- El **componente social** de iNaturalist (seguimiento, votos, comentarios), adaptado al ámbito medioambiental con la distinción de comentarios oficiales.
- La **robustez offline** de Epicollect5, incorporada mediante Service Worker y IndexedDB.
- Un conjunto propio de **mecanismos de seguridad avanzados** (2FA TOTP, *audit log*, Cloudflare Turnstile) que no aparecen en ninguna de las aplicaciones analizadas.

Este posicionamiento, junto con la prioridad dada a la accesibilidad (WCAG 2.1 AA) y a la transparencia (código abierto, declaración explícita del uso de IA), constituye la propuesta de valor diferencial de EcoAlerta.

Chapter 3

Análisis y diseño

Este capítulo describe el proceso de análisis de requisitos y el diseño técnico de EcoAlerta. Se presentan la metodología de desarrollo, los actores del sistema, los requisitos funcionales y no funcionales, la arquitectura de software, el modelo de datos, el diseño de la interfaz de programación de aplicaciones (API) REST y el diseño preliminar de la interfaz de usuario. El contenido de este capítulo constituye la base sobre la que se ha construido la implementación descrita en el Capítulo 4.

3.1 Metodología de desarrollo

El desarrollo de EcoAlerta se ha organizado siguiendo un enfoque **iterativo e incremental** estructurado en seis sprints de una semana cada uno. Esta elección se justifica por tres motivos prácticos: la naturaleza cambiante de los requisitos derivada de las tutorías periódicas con el tutor, la posibilidad de obtener un producto funcional temprano para recibir realimentación, y la necesidad de ajustar el alcance de cada iteración al tiempo disponible del estudiante.

Sobre este esquema de sprints se ha aplicado la metodología **Spec-Driven Development (SDD)**, concretamente en su implementación *SpecKit* [?], un flujo que estructura cada iteración en cuatro fases claramente separadas:

1. **Specify.** Redacción de la especificación funcional del sprint, enumerando los requisitos que se implementarán y los criterios de aceptación.
2. **Plan.** Descomposición de la especificación en tareas concretas, con dependencias y estimación.
3. **Implement.** Ejecución de las tareas, principalmente asistida por un entorno de desarrollo con agentes de inteligencia artificial (*Google Antigravity*).
4. **Verify.** Validación mediante pruebas unitarias, de integración y extremo a extremo, y verificación del cumplimiento de los criterios de aceptación.

La elección de SDD responde a una necesidad específica del proyecto: al emplearse asistencia de inteligencia artificial en la fase de implementación (véase la sección 1.3 del Capítulo 1), es fundamental disponer de una especificación escrita y verificable que sirva tanto de guía al agente como de referencia para auditar su trabajo. SDD proporciona exactamente ese andamiaje. El documento maestro que consolida todas las especificaciones, `EcoAlerta_Development_Spec_v1.md`, se ha mantenido versionado en el repositorio Git junto al código y constituye la fuente de verdad del proyecto.

Se han celebrado tutorías con el tutor académico con periodicidad aproximadamente quincenal, cuyas actas se encuentran en `docs/actas-tutorias/` del repositorio. Cada sprint se ha cerrado con una retrospectiva breve documentada en `docs/retrospectivas/`.

3.2 Actores del sistema

El análisis de stakeholders derivado del estudio del estado del arte (Capítulo 2) ha identificado cinco actores que interactúan con el sistema, resumidos en la Tabla 3.1.

Table 3.1: Actores del sistema EcoAlerta.

Actor	Descripción	Autenticación
Ciudadano anónimo	Visitante sin cuenta. Puede consultar el mapa público y el detalle de incidencias, pero no puede crear reportes, votar ni comentar.	Ninguna
Ciudadano registrado	Usuario con cuenta verificada. Puede crear incidencias, votar, comentar y seguir reportes. Dispone de perfil social.	Email + contraseña (JWT)
Administrador municipal	Gestiona las incidencias del sistema: validación, asignación a entidades responsables y cambios de estado. Accede al <i>dashboard</i> de estadísticas.	Email + contraseña (rol admin)
Entidad responsable	Organismo encargado de atender incidencias: Ayuntamiento, SEPRONA, Bomberos, Policía Local u ONG. Recibe las incidencias asignadas y actualiza su estado.	Email + contraseña (rol entity)
Sistema	Actor interno no humano. Envía notificaciones automáticas, escala incidencias en <i>timeout</i> y registra eventos en el <i>audit log</i> .	Interno

La distinción entre *ciudadano anónimo* y *ciudadano registrado* es una decisión de diseño deliberada: permite maximizar el valor público de la plataforma (cualquiera puede consultar el mapa como herramienta de transparencia cívica), a la vez que reserva las acciones con responsabilidad (crear, votar, comentar) a usuarios trazables para prevenir abusos.

3.3 Requisitos funcionales

A continuación se presentan los 46 requisitos funcionales (RF) extraídos del análisis, agrupados por módulo. La prioridad se expresa en tres niveles: *Alta* (crítico para el sistema mínimo viable), *Media* (importante para la experiencia de usuario) y *Baja* (deseable pero postergable).

3.3.1 Módulo de autenticación

Table 3.2: Requisitos funcionales del módulo de autenticación.

ID	Prioridad	Descripción	Actor
RF-01	Alta	Registro de usuarios con email y contraseña.	Ciudadano
RF-02	Alta	Inicio de sesión con tokens JWT <i>access</i> y <i>refresh</i> .	Todos
RF-03	Alta	Acceso anónimo con funcionalidades restringidas (solo lectura de mapa y listado).	Anónimo
RF-04	Alta	Soporte de roles: <i>citizen</i> , <i>admin</i> , <i>entity</i> , <i>moderator</i> .	Sistema
RF-05	Baja	Recuperación de contraseña por email (limitada a la disponibilidad de SMTP).	Ciudadano
RF-06	Media	Edición del perfil (nombre, avatar, bio).	Ciudadano
RF-07	Media	Perfil público con las incidencias reportadas por el usuario.	Ciudadano

3.3.2 Módulo de incidencias

Table 3.3: Requisitos funcionales del módulo de incidencias.

ID	Prioridad	Descripción	Actor
RF-10	Alta	Creación de incidencias con título, descripción, categoría, fotos (1–5) y ubicación GPS.	Ciudadano reg.
RF-11	Alta	Captura automática de la ubicación GPS al crear la incidencia.	Sistema
RF-12	Alta	Selección manual de ubicación en el mapa si el GPS no está disponible.	Ciudadano
RF-13	Alta	Clasificación por categoría (12 categorías predefinidas).	Sistema
RF-14	Alta	Asignación de severidad: <i>leve</i> , <i>moderado</i> , <i>grave</i> , <i>crítico</i> .	Ciudadano/Admin
RF-15	Alta	Adjuntar entre 1 y 5 fotografías por incidencia (máx. 10 MB cada una).	Ciudadano

ID	Prioridad	Descripción	Actor
RF-16	Media	Generación automática de <i>thumbnails</i> de las fotos.	Sistema
RF-17	Media	Edición o eliminación de incidencias propias sólo si el estado es <i>pending</i> .	Ciudadano
RF-18	Media	Historial de cambios de estado almacenado en <i>incident_status_history</i> .	Sistema
RF-19	Baja	Guardado de borradores de incidencia con capacidad <i>offline</i> (PWA).	Ciudadano

3.3.3 Módulo de mapa interactivo

Table 3.4: Requisitos funcionales del módulo de mapa interactivo.

ID	Prioridad	Descripción	Actor
RF-20	Alta	Mapa interactivo con todas las incidencias geolocalizadas.	Todos
RF-21	Alta	Agrupación de incidencias cercanas en <i>clusters</i> al disminuir el zoom.	Sistema
RF-22	Alta	Filtros por categoría, severidad, estado, fecha y distancia.	Todos
RF-23	Alta	Marcadores coloreados según severidad (verde, amarillo, naranja, rojo).	Sistema
RF-24	Media	Búsqueda por dirección o nombre de lugar (geocoding).	Todos
RF-25	Alta	Detalle de incidencia al pulsar el marcador.	Todos
RF-26	Media	Búsquedas geoespaciales por radio (<i>ST_DWithin</i>).	Sistema

3.3.4 Módulo de administración y delegación

Table 3.5: Requisitos funcionales del módulo de administración.

ID	Prioridad	Descripción	Actor
RF-30	Alta	Panel de administración con <i>dashboard</i> de estadísticas agregadas.	Admin
RF-31	Alta	Cambio de estado de la incidencia (pending → validated → in_progress → resolved/rejected).	Admin
RF-32	Alta	Asignación de incidencias a entidades responsables.	Admin
RF-33	Alta	La entidad recibe notificación de la asignación y puede actualizar el estado.	Entidad
RF-34	Media	Escalado automático de incidencias no atendidas tras un <i>timeout</i> configurable.	Sistema
RF-35	Media	Estadísticas: por categoría, por estado, por zona, tiempo medio de resolución.	Admin
RF-36	Media	Gestión de categorías y entidades responsables.	Admin
RF-37	Baja	Marcado de incidencias como <i>duplicadas</i> con vinculación.	Admin

3.3.5 Módulo social y notificaciones

Table 3.6: Requisitos funcionales del módulo social y de notificaciones.

ID	Prioridad	Descripción	Actor
RF-40	Alta	Votación/apoyo a incidencias de otros usuarios.	Ciudadano
RF-41	Alta	Comentarios en incidencias, con distintivo para los oficiales.	Ciud./Admin/Ent.
RF-42	Alta	Notificación al autor y seguidores cuando cambia el estado.	Sistema
RF-43	Media	Seguir incidencias para recibir actualizaciones (<i>incident_follows</i>).	Ciudadano
RF-44	Media	Perfil de usuario con incidencias reportadas, votos y estadísticas.	Ciudadano

ID	Prioridad	Descripción	Actor
RF-45	Media	Notificaciones <i>push</i> mediante Service Worker (PWA).	Sistema
RF-46	Baja	Notificaciones por email para cambios de estado importantes.	Sistema

3.4 Requisitos no funcionales

Los requisitos no funcionales (RNF) definen atributos de calidad que el sistema debe satisfacer más allá de su funcionalidad explícita. La Tabla 3.7 los enumera agrupados por categoría. Todos son verificables mediante métricas objetivas, lo que permite su validación en el Capítulo 5.

Table 3.7: Requisitos no funcionales del sistema.

ID	Categoría	Descripción
RNF-01	Rendimiento	Tiempo de respuesta de la API inferior a 500 ms para el 95 % de las peticiones.
RNF-02	Rendimiento	Soporte para al menos 100 usuarios simultáneos sin degradación.
RNF-03	Usabilidad	Diseño <i>responsive</i> : móvil (320 px), tablet (768 px) y escritorio (≥ 1024 px).
RNF-04	Portabilidad	PWA instalable con <code>manifest.json</code> y Service Worker.
RNF-05	Seguridad	Contraseñas almacenadas con <i>bcrypt</i> (mínimo 10 rondas).
RNF-06	Seguridad	<i>Rate limiting</i> en todos los endpoints de la API.
RNF-07	Seguridad	HTTPS obligatorio en entorno de producción.
RNF-08	Calidad	Cobertura de pruebas unitarias superior al 70 % en el <i>backend</i> .
RNF-09	Calidad	Pruebas E2E para los tres flujos principales (reporte, mapa, admin).
RNF-10	Mantenibilidad	Despliegue reproducible con un único comando (<code>docker compose up</code>).
RNF-11	Mantenibilidad	Documentación Swagger/OpenAPI de la API REST.

ID	Categoría	Descripción
RNF-12	Accesibilidad	Cumplimiento WCAG 2.1 nivel AA (contraste, navegación por teclado, lectores).

3.5 Casos de uso

La Figura 3.1 presenta el diagrama UML de casos de uso del sistema, agrupados por módulos funcionales. Las relaciones «requiere» indican casos de uso que únicamente pueden ejecutarse por un actor autenticado, mientras que «include» señala casos de uso que son invocados como efecto secundario de otros (por ejemplo, toda validación administrativa incluye automáticamente el envío de una notificación al autor).

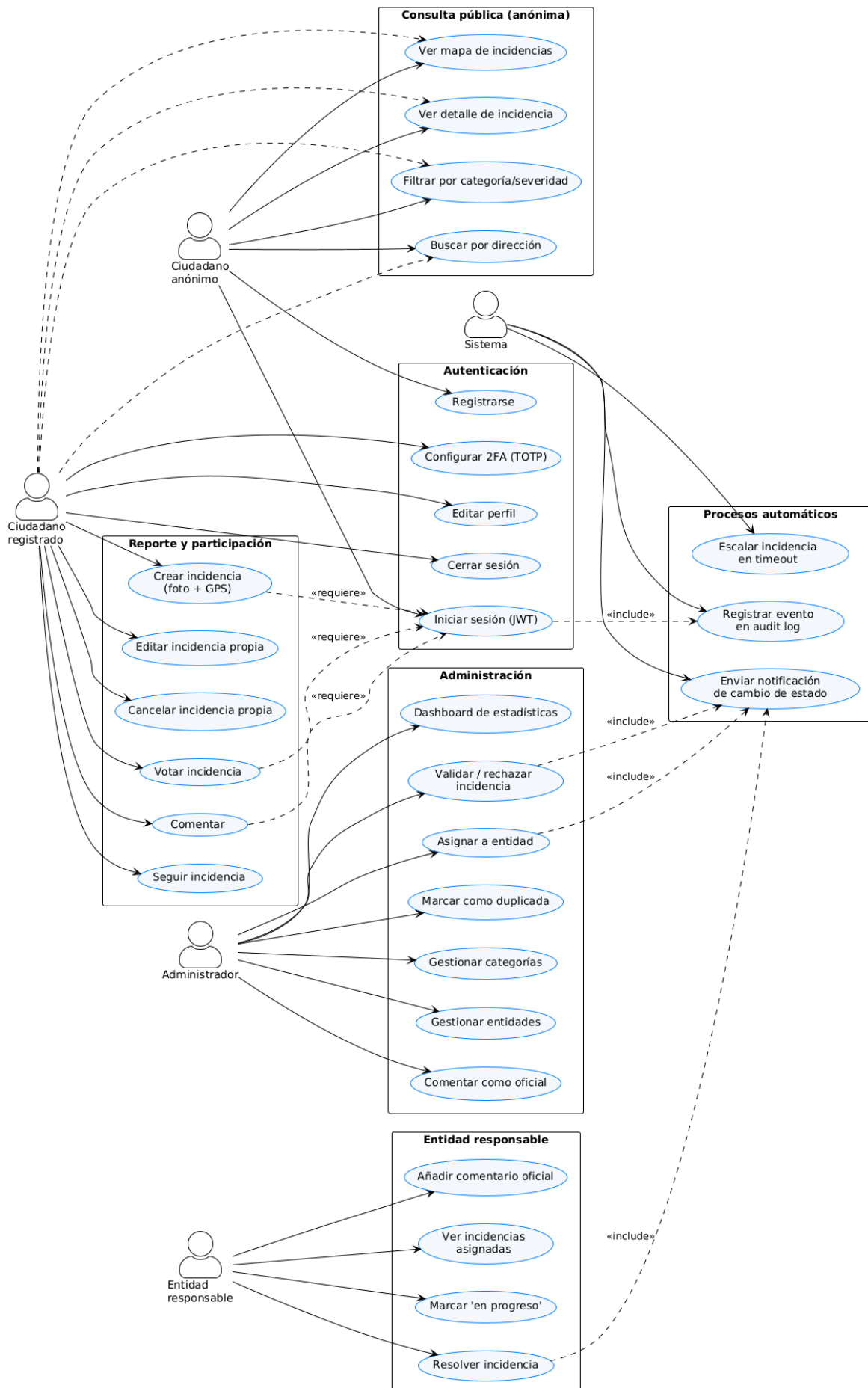


Figure 3.1: Diagrama de casos de uso de EcoAlerta. Fuente: elaboración propia a partir de los requisitos RF-01–RF-46.

Cabe destacar la presencia del actor *Sistema*, que agrupa los procesos automáticos ejecutados sin intervención humana directa: envío de notificaciones tras cambios de estado (RF-42), escalado por *timeout* (RF-34) y registro de eventos de seguridad en el *audit log*.

3.6 Arquitectura del sistema

EcoAlerta sigue una arquitectura **cliente-servidor de tres capas** con separación clara de responsabilidades:

1. **Capa de presentación.** Aplicación React 18 con Tailwind CSS, servida como *Single Page Application* (SPA). Incluye el mapa interactivo de Leaflet, formularios de reporte y el panel de administración. La aplicación es instalable como PWA gracias al Service Worker y al `manifest.json`.
2. **Capa de lógica de negocio.** API REST implementada con Node.js 20 y Express 4, organizada en controladores, servicios, *middlewares* y validadores. Expone documentación interactiva en `/api/docs` mediante Swagger UI.
3. **Capa de datos.** PostgreSQL 16 con la extensión PostGIS 3.4, que provee tipos geométricos nativos y funciones espaciales (`ST_DWithin`, `ST_Distance`, índices GIST).

La separación en capas se complementa con un servidor *reverse proxy* nginx que termina las conexiones HTTPS, sirve los ficheros estáticos del *frontend* y redirige las rutas `/api/*` al contenedor del *backend*. Todo el conjunto se despliega mediante Docker Compose, lo que garantiza la reproducibilidad del entorno (RNF-10) y facilita la demostración durante la defensa.

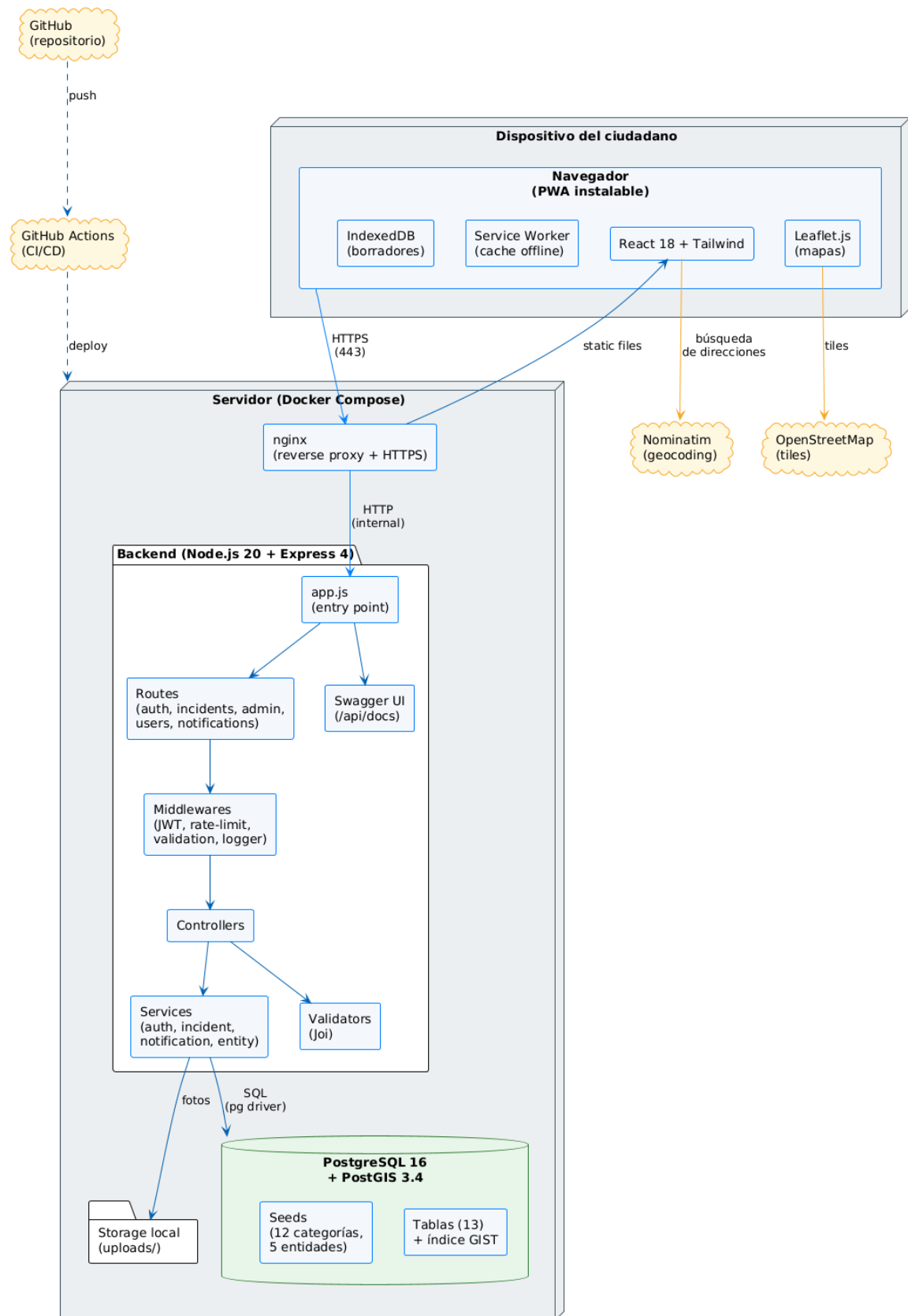


Figure 3.2: Diagrama de arquitectura de componentes de EcoAlerta.

El diseño contempla también la integración con servicios externos gratuitos: los

tiles cartográficos se sirven desde OpenStreetMap y el *geocoding* inverso se realiza contra Nominatim. Esta elección evita dependencias de proveedores con API de pago (Google Maps, Mapbox) y simplifica el modelo de negocio descrito en el Anexo C.

3.6.1 Stack tecnológico y justificación

La Tabla 3.8 resume las tecnologías seleccionadas, con la justificación concreta de cada elección. El tutor dio libertad total en la elección del *stack* (véase acta de la tutoría del 24 de marzo de 2026), siempre que la decisión quedara razonada en la memoria.

Table 3.8: Stack tecnológico empleado y justificación de cada elección.

Capa	Tecnología	Justificación
Frontend	React 18 + Tailwind CSS 3	Ecosistema maduro, renderizado eficiente con <i>Virtual DOM</i> , comunidad amplia y compatibilidad directa con Leaflet. Tailwind permite diseño responsive (RNF-03) sin abandonar la semántica HTML.
Mapas	Leaflet.js + OpenStreetMap	Biblioteca <i>open source</i> sin necesidad de API key ni cuota. La alternativa Google Maps implicaba coste y dependencia comercial.
PWA	Service Worker + <code>manifest.json</code>	Permite instalación en móvil y caché <i>offline</i> (RNF-04, RF-19) sin necesidad de publicar en tiendas de aplicaciones.
Backend	Node.js 20 LTS + Express 4	Ejecución asíncrona no bloqueante idónea para cargas I/O (BD, subida de fotos). <i>Middleware</i> ecosystem amplio. LTS asegura soporte hasta abril 2026.
Base de datos	PostgreSQL 16 + PostGIS 3.4	PostGIS aporta tipos geométricos nativos y funciones como <code>ST_DWithin</code> (RF-26), críticas para el mapa. Alternativas NoSQL como MongoDB carecen de índices espaciales GIST con las mismas garantías.

Capa	Tecnología	Justificación
Autenticación	JWT (access + refresh) + <i>bcryptjs</i>	Stateless, compatible con PWA y escalable horizontalmente. <i>Bcrypt</i> con 10 rondas satisface RNF-05.
Contenedores	Docker + Docker Compose	Entorno reproducible con un comando (RNF-10). Facilita la demostración durante la defensa.
CI/CD	GitHub Actions	Integración gratuita con el repositorio. Ejecuta <i>lint</i> y pruebas en cada <i>push</i> .
Pruebas	Jest + Supertest + Playwright	Jest y Supertest cubren unit e integration en el <i>backend</i> (RNF-08). Playwright gestiona E2E (RNF-09) y ofrece mejor integración con MCPs que Cypress.
Logging	Winston	Logging estructurado JSON, compatible con agregadores de logs externos.
Documentación API	Swagger (swagger-jsdoc + swagger-ui-express)	Satisface RNF-11. La documentación se genera a partir de anotaciones en el propio código, evitando desincronización.
Control de versiones	Git + GitHub (Conventional Commits)	Historial auditable y semántico, útil durante la defensa y para generar automáticamente el CHANGELOG.md.

3.7 Modelo de datos

El modelo de datos consta de **trece entidades** diseñadas para soportar el reporte, la gestión y el seguimiento de incidencias medioambientales con datos geoespaciales y un *audit trail* completo. La Figura 3.3 muestra el diagrama entidad-relación.

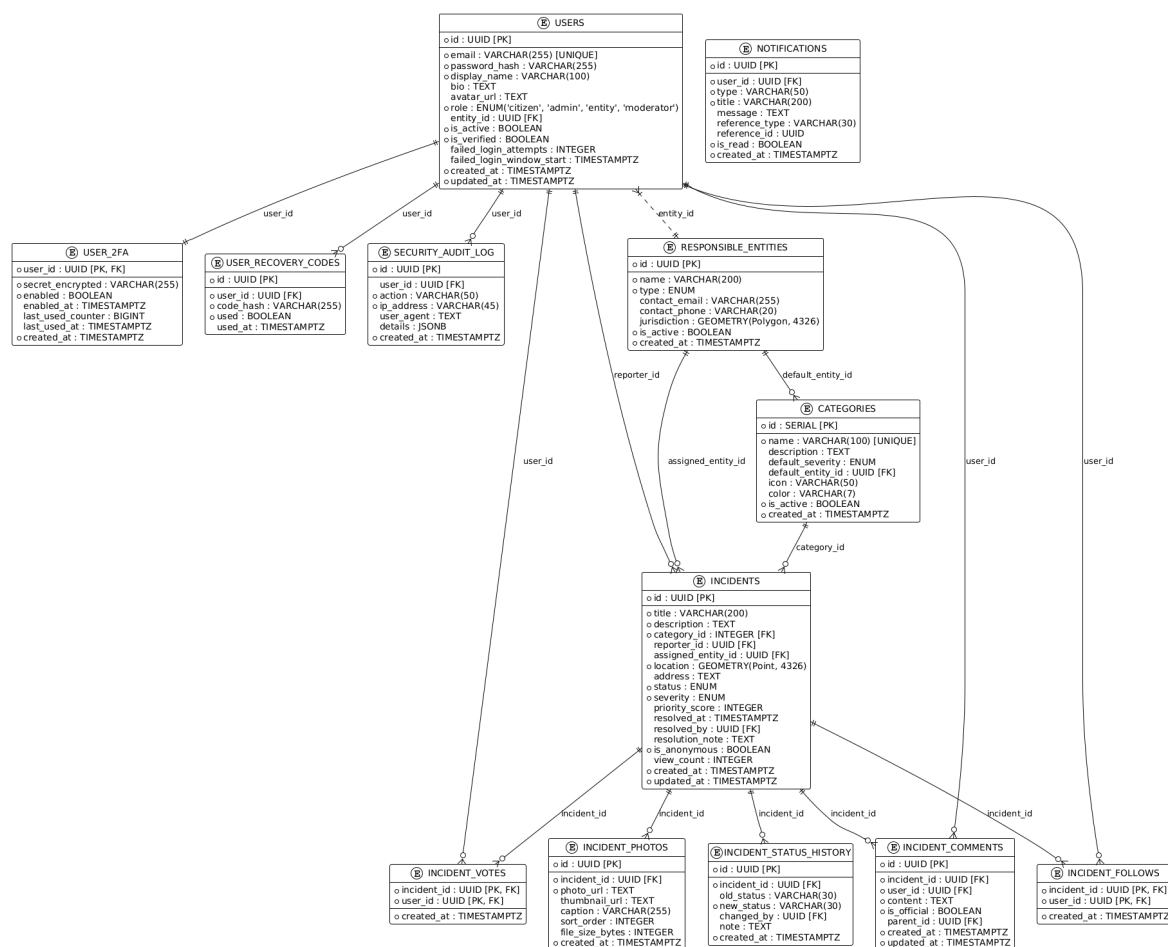


Figure 3.3: Modelo entidad-relación de EcoAlerta (13 tablas).

Las entidades pueden agruparse en cuatro subsistemas lógicos:

Identidad y seguridad:

users, user_2fa, user_recovery_codes, security_audit_log.

Catálogos maestros:

categories, responsible_entities.

Núcleo de incidencias:

incidents, incident_photos, incident_comments, incident_status_history.

Interacción social y notificaciones:

incident_votes, incident_follows, notifications.

3.7.1 Descripción de las entidades principales

users. Almacena la información de los usuarios registrados. Los campos role, entity_id, is_active e is_verified permiten implementar el control de acceso basado en roles (RF-04). Los contadores failed_login_attempts y failed_login_window_start soportan un mecanismo de *rate-limit* por usuario que refuerza RNF-06.

user_2fa, user_recovery_codes. Entidades añadidas en el Sprint 6 de *security hardening*. Permiten habilitar autenticación de doble factor por TOTP y almacenar códigos de recuperación cifrados, ampliando el alcance inicial del RF-02.

security_audit_log. Registra eventos de seguridad (inicios de sesión, cambios de rol, cambios de contraseña) con dirección IP, *user agent* y metadatos en JSONB. No se borra nunca: actúa como *audit trail* completo del sistema.

responsible_entities. Catálogo de organismos a los que pueden asignarse incidencias. El campo *jurisdiction* es de tipo `GEOMETRY(Polygon, 4326)`, lo que permite en el futuro asignación automática por intersección geoespacial entre la ubicación de la incidencia y la jurisdicción de la entidad.

categories. Define las 12 categorías predefinidas de incidencias (vertidos, contaminación, incendio forestal, fauna, etc.) con su severidad por defecto y entidad responsable sugerida, reduciendo la fricción en el reporte ciudadano.

incidents. Entidad central del sistema. El campo *location* es `GEOMETRY(Point, 4326)` e incluye un índice GIST que hace posible consultas geoespaciales de bajo coste (RF-26). El *priority_score* es un entero derivado de severidad y número de votos que permite al *dashboard* administrativo ordenar por urgencia (RF-35).

incident_photos, incident_comments, incident_votes, incident_follows. Relaciones 1:N (fotos, comentarios) y N:M (votos, seguimientos) del núcleo social. Las tablas de votos y seguimientos usan clave primaria compuesta (*incident_id*, *user_id*) que garantiza unicidad natural sin necesidad de índice adicional.

incident_status_history. Historial inmutable de cambios de estado (RF-18). Cada transición del flujo descrito en la sección 3.8 inserta una fila con estado anterior, nuevo, usuario responsable y nota opcional. Es la base de la trazabilidad del sistema.

notifications. Notificaciones por usuario. El campo *reference_type* identifica el tipo de entidad referenciada (*incident*, *comment*, etc.) y *reference_id* actúa como *soft foreign key*, permitiendo polimorfismo sin sacrificar la integridad referencial del resto del modelo.

3.7.2 Consideraciones de rendimiento geoespacial

Las consultas más frecuentes del sistema son de naturaleza geoespacial: «incidencias en un radio de X km del punto Y». PostgreSQL + PostGIS permite resolverlas eficientemente mediante el operador `ST_DWithin` combinado con un índice GIST sobre la columna `location`:

```
CREATE INDEX idx_incidents_location ON incidents USING GIST (  
    location);  
  
SELECT id, title, severity  
FROM incidents  
WHERE ST_DWithin(location::geography,  
                 ST_MakePoint(:lng, :lat)::geography,  
                 :radius_meters);
```

La proyección a tipo `geography` antes del `ST_DWithin` garantiza que la distancia se evalúe sobre la superficie terrestre real (metros) y no sobre el plano cartesiano (grados), lo que sería incorrecto fuera del ecuador.

3.8 Flujo de estados de una incidencia

Cada incidencia atraviesa un ciclo de vida acotado por un conjunto finito de estados, implementado como un `ENUM` en PostgreSQL. La Figura 3.4 presenta el diagrama UML de estados.

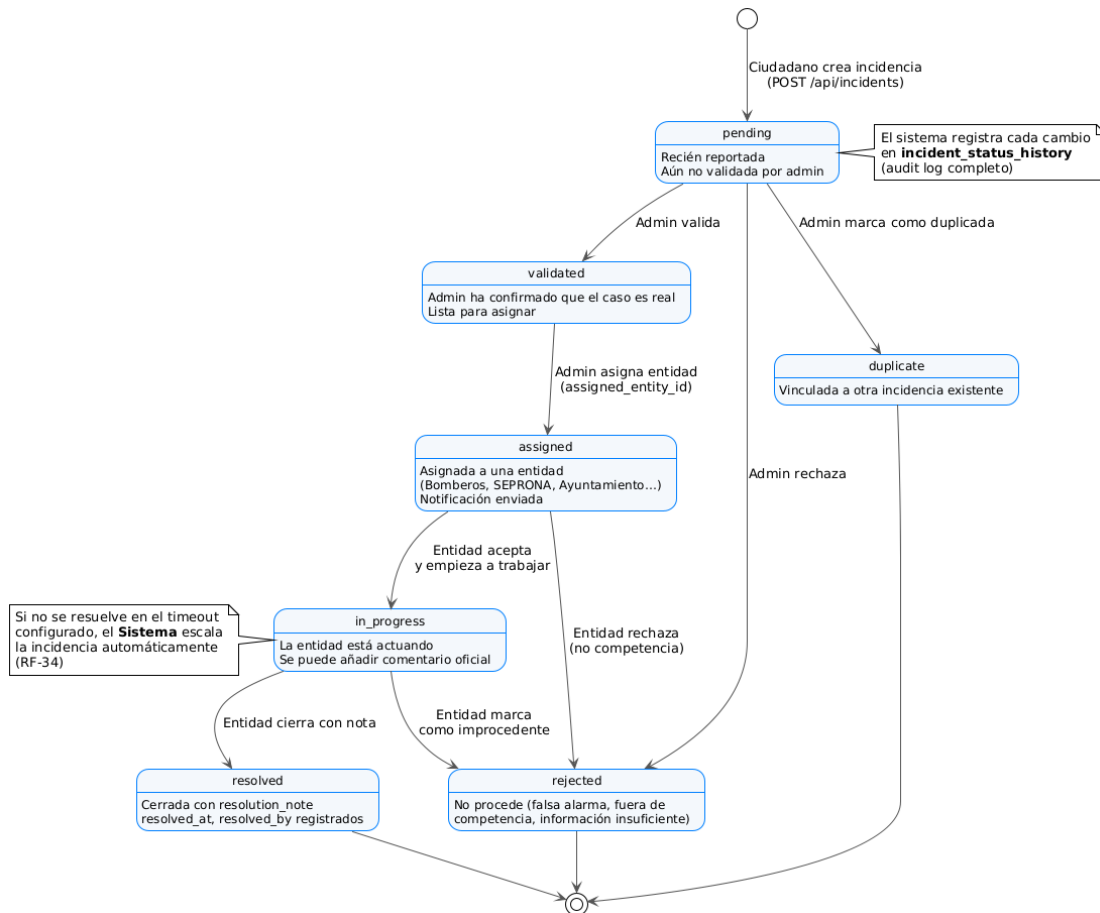


Figure 3.4: Diagrama UML de estados de una incidencia. Cada transición se almacena en `incident_status_history` con *timestamp* y usuario responsable.

El estado inicial de toda incidencia creada por un ciudadano es `pending`. A partir de ahí, un administrador puede validarla, rechazarla o marcarla como duplicada. Tras la validación, la asignación a una entidad responsable genera la transición a `assigned`, momento en que se dispara una notificación automática (RF-42). La entidad responsable confirma el inicio del trabajo (`in_progress`) y, tras actuar, cierra la incidencia con `resolved` y una nota descriptiva.

Como excepciones al flujo principal, una entidad puede rechazar una incidencia si constata que no es de su competencia. Adicionalmente, el requisito RF-34 contempla el escalado automático por el Sistema cuando una incidencia en `in_progress` supera un *timeout* configurable sin avances.

3.9 Diseño de la API REST

La API expone un total de **35 endpoints** organizados en seis recursos, siguiendo las convenciones REST y prefijados con la versión `/api/v1/` para facilitar una futura evolución. La documentación interactiva Swagger se expone en `/api/docs`.

Table 3.9: Endpoints principales de la API REST.

Método	Ruta	Descripción
<i>Recurso: autenticación</i>		
POST	/auth/register	Registro de ciudadano.
POST	/auth/login	Login, devuelve <i>access</i> y <i>refresh token</i> .
POST	/auth/refresh	Renovación del <i>access token</i> .
GET	/auth/me	Perfil del usuario autenticado.
PUT	/auth/me	Actualización de perfil.
<i>Recurso: incidencias</i>		
GET	/incidents	Listado paginado con filtros.
GET	/incidents/:id	Detalle con fotos y comentarios.
POST	/incidents	Creación (autenticado, <i>multipart</i>).
PUT	/incidents/:id	Edición (autor o admin).
DELETE	/incidents/:id	Baja (autor o admin).
GET	/incidents/nearby	Geoespacial (?lat, lng, radius).
GET	/incidents/:id/history	Historial de cambios de estado.
POST	/incidents/:id/vote	Votar o apoyar.
POST	/incidents/:id/follow	Seguir incidencia.
POST	/incidents/:id/comments	Añadir comentario.
<i>Recurso: administración</i>		
GET	/admin/dashboard	Estadísticas agregadas.
GET	/admin/incidents	Listado administrativo con filtros avanzados.
PUT	/admin/incidents/:id/status	Cambio de estado.
PUT	/admin/incidents/:id/assign	Asignación a entidad.
GET	/admin/users	Listado de usuarios.
PUT	/admin/users/:id/role	Cambio de rol.
GET	/admin/entities	Listado de entidades responsables.
POST	/admin/entities	Creación de entidad.
PUT	/admin/entities/:id	Edición de entidad.
<i>Recurso: usuarios (perfil público)</i>		
GET	/users/:id	Perfil público.
GET	/users/:id/incidents	Incidencias del usuario.
GET	/users/:id/stats	Estadísticas del usuario.
<i>Recurso: notificaciones</i>		
GET	/notifications	Notificaciones del usuario.
PUT	/notifications/:id/read	Marcar como leída.

Método	Ruta	Descripción
PUT	/notifications/read-all	Marcar todas como leídas.
<i>Salud del sistema</i>		
GET	/health	<i>Health check.</i>

La autenticación se implementa con el esquema Bearer sobre JWT. El *access token* tiene una validez corta (15 minutos) mientras que el *refresh token* tiene validez extendida (7 días) y rotación en cada uso. El *middleware* de verificación inspecciona la firma y comprueba la vigencia antes de poblar `req.user` para los controladores.

3.10 Diseño de la interfaz de usuario

El diseño de la interfaz sigue tres principios rectores derivados del análisis de aplicaciones de referencia (Capítulo 2):

1. **Mobile-first.** El caso de uso principal (reportar una incidencia sobre el terreno) ocurre desde un dispositivo móvil. Los *breakpoints* de Tailwind (sm, md, lg, xl) se utilizan para adaptar progresivamente la interfaz a pantallas mayores (RNF-03).
2. **Mapa como eje central.** La pantalla principal es el mapa, no una lista textual. El mapa transmite de forma inmediata densidad y distribución geográfica del problema, lo que es coherente con la naturaleza cívica del producto.
3. **Accesibilidad por diseño.** Contrastes verificados con la herramienta *Contrast Checker*, navegación completa por teclado y etiquetas ARIA en todos los controles interactivos (RNF-12).

Las pantallas clave del sistema son: mapa principal, formulario de reporte, detalle de incidencia, panel de administración y perfil de usuario. Las capturas de la aplicación real se incluyen en el Capítulo 4 (sección 4.7) y en el manual de usuario del Anexo B.

Chapter 4

Implementación

Este capítulo documenta el desarrollo efectivo de EcoAlerta. Se describe la estructura del repositorio, la configuración del entorno, la organización en sprints (con las decisiones técnicas tomadas en cada uno), los extractos de código más significativos y las prácticas de integración continua empleadas. El contenido se apoya en el diseño presentado en el Capítulo 3 y prepara el terreno para la validación recogida en el Capítulo 5.

4.1 Entorno de desarrollo y herramientas

El desarrollo se ha realizado sobre Windows 11 con PowerShell como *shell* principal. Las herramientas empleadas se resumen en la Tabla 4.1.

Table 4.1: Herramientas del entorno de desarrollo.

Categoría	Herramienta
IDE principal	Google Antigravity (<i>fork</i> de VS Code con agentes de IA)
Gestión de versiones	Git + GitHub, convenciones <i>Conventional Commits</i>
Contenedores	Docker Desktop + Docker Compose v2
Base de datos local	PostgreSQL 16 + PostGIS 3.4 (contenedor)
Node.js	Node.js 20 LTS, npm
Pruebas <i>backend</i>	Jest 29 + Supertest 7
Pruebas <i>frontend</i> E2E	Playwright 1.38 (vía MCP de Antigravity)
Bibliografía	Zotero (exporta BibLaTeX)
Documentación API	Swagger UI expuesto en <code>/api/docs</code>
MCPs empleados	GitHub, Context7, Playwright, Figma

La elección de Antigravity como IDE se justifica por dos factores: la integración nativa con agentes de IA (lo que hace natural el flujo SDD descrito en la sección 3.1) y

la compatibilidad con el ecosistema MCP, que aporta capacidades como la ejecución de pruebas Playwright sin salir del editor.

4.2 Estructura del repositorio

El repositorio se organiza siguiendo una separación estricta entre código, documentación, infraestructura y pruebas:

```
tfg-medioambiente-ua/
|-- .github/workflows/      # CI/CD (GitHub Actions)
|-- database/               # init.sql, seeds, migraciones
|-- docs/                   # Documentacion del TFG
|   |-- memoria/           #   - Fuentes LaTeX de la memoria
|   |-- diagramas/         #   - Diagramas PlantUML
|   |-- actas-tutorias/    #   - Actas de las tutorias
|   |-- retrospectivas/    #   - Retrospectivas de sprint
|   '-- sprints/           #   - Specs SDD por sprint
|-- nginx/                  # Configuracion reverse proxy
|-- scripts/                # Scripts auxiliares de gestion
|-- src/
|   |-- backend/            # API Node.js + Express
|   '-- frontend/          # SPA React 18 + Tailwind + PWA
|-- docker-compose.dev.yml  # Entorno de desarrollo
|-- docker-compose.yml     # Entorno de produccion
|-- Makefile                # Atajos (make dev, make test, make prod)
|-- CHANGELOG.md            # Historial de versiones
|-- README.md               # Documentacion de arranque
'-- EcoAlerta_Development_Spec_v1.md # Spec maestra SDD
```

Dentro de `src/backend` se ha empleado la organización en capas *routes* → *controllers* → *services* propia de aplicaciones Express profesionales: las rutas definen los *endpoints*, los controladores orquestan la petición/respuesta, y los servicios encapsulan la lógica de negocio y las consultas a base de datos. Los *validators* se apoyan en `express-validator` para validar el cuerpo y parámetros de entrada antes de que la petición alcance al controlador.

En `src/frontend` se sigue la estructura estándar de una aplicación React por *feature*: `components/` (común, autenticación, incidencias, mapa, seguridad, administración), `pages/` (una por ruta), `context/` (contextos globales, principalmente autenticación), `hooks/` (lógica reutilizable), `services/` (cliente HTTP con Axios) y `utils/`.

4.3 Desarrollo por sprints

El desarrollo se organizó en seis sprints, descritos a continuación. Cada uno de ellos se abrió con una especificación detallada (almacenada en `docs/sprints/`) y se cerró con una retrospectiva breve. La Tabla 4.2 resume el objetivo principal y los entregables clave de cada sprint.

Table 4.2: Resumen de los seis sprints de desarrollo.

Sprint	Tema	Entregables principales
1	Autenticación + BD	Esquema PostGIS, JWT dual token, roles, middlewares de seguridad
2	CRUD incidencias	Creación con foto y GPS, validaciones, <i>thumbnails</i> con Sharp
3	Mapa interactivo	Leaflet + <i>clusters</i> , filtros, búsqueda geoespacial (ST_DWithin)
4	Admin + delegación	Dashboard, cambio de estado, asignación a entidades, notificaciones
5	Testing + PWA + <i>polish</i>	Jest, Playwright, Service Worker, <code>manifest.json</code> , Swagger
6	<i>Security hardening</i>	2FA TOTP, Cloudflare Turnstile, <i>audit log</i> , <i>rate limiting</i> por usuario

4.3.1 Sprint 1 — Autenticación y base de datos

El primer sprint estableció los cimientos del sistema: el esquema de la base de datos y el subsistema de autenticación.

La base de datos se inicializa mediante un único `init.sql` ejecutado automáticamente por el contenedor de PostgreSQL la primera vez que se levanta. El script habilita la extensión PostGIS, crea las trece tablas descritas en la sección 3.7, define los tipos ENUM (`user_role`, `incident_status`, `incident_severity`), crea el índice GIST sobre la columna `incidents.location` e inserta las 12 categorías de incidencias y las 5 entidades responsables por defecto.

El subsistema de autenticación se implementa sobre JSON Web Tokens con un esquema de *access token* corto (15 minutos) y *refresh token* extendido (7 días). La función `generateTokens` del servicio, que firma ambos tokens con el secreto de entorno, ilustra esta aproximación:

```
const generateTokens = (user) => {
  const payload = { id: user.id, email: user.email, role: user.role };
  const accessToken = jwt.sign(payload, process.env.JWT_SECRET, {
```

```
    expiresIn: '15m',
  });
  const refreshToken = jwt.sign(payload, process.env.JWT_SECRET, {
    expiresIn: '7d',
  });
  return { accessToken, refreshToken };
};
```

Listing 4.1: Generación de tokens JWT (auth.service.js).

Las contraseñas se almacenan en la columna `password_hash` después de aplicar *bcrypt* con 10 rondas (`SALT_ROUNDS = 10`), satisfaciendo el requisito RNF-05. El *middleware* `authenticate` verifica el token en cada petición protegida y decora `req.user` con el *payload*; una variante `optionalAuth` permite que los *endpoints* de lectura pública funcionen también sin autenticación, dejando `req.user = null` (soporta RF-03).

Para el control de acceso basado en roles se implementó un patrón *factory* de *middlewares*:

```
const requireRole = (roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({
      success: false,
      error: 'Acceso restringido. Roles permitidos: ${roles.join(', ')}',
    });
  }
  next();
};

const requireAdmin = requireRole(['admin']);
const requireEntity = requireRole(['entity']);
```

Listing 4.2: Control de acceso por rol (auth.middleware.js).

4.3.2 Sprint 2 — CRUD de incidencias, fotos y GPS

El segundo sprint dio soporte al flujo principal del sistema: crear una incidencia con foto y ubicación GPS. Desde el punto de vista del *backend*, se implementaron los *endpoints* descritos en la Tabla 3.9 bajo el recurso `/incidents`, con particular atención a tres aspectos:

1. **Subida multipart.** El *middleware* `multer` gestiona la subida de fotografías (hasta 5 por incidencia, máximo 10 MB cada una). El control de tamaño y tipo MIME se realiza antes de llegar al controlador.
2. **Procesamiento de imágenes.** La librería `sharp` genera *thumbnails* JPEG (400 px

de ancho, calidad 80) a partir de cada foto original, cumpliendo RF-16 sin cargar proceso al cliente.

3. **Almacenamiento geoespacial.** La ubicación se guarda con `ST_SetSRID(ST_MakePoint(lng, lat), 4326)` para garantizar que el SRID (*Spatial Reference System Identifier*) quede explícitamente asociado al punto. Esto evita ambigüedades futuras en consultas que combinen distintas referencias cartográficas.

En el *frontend* se desarrolló la página `CreateIncidentPage` con un formulario de varios pasos: captura automática de geolocalización (con *fallback* a selección manual sobre el mapa), selección de categoría y severidad, subida de fotos (con previsualización) y descripción. El formulario guarda un borrador local con IndexedDB si se pierde la conexión, anticipando la funcionalidad *offline* del Sprint 5.

4.3.3 Sprint 3 — Mapa interactivo y filtros geoespaciales

El tercer sprint implementó el corazón visual de la aplicación: el mapa interactivo con todas las incidencias. Se empleó `react-leaflet` sobre tiles de OpenStreetMap. Los marcadores se colorean según la severidad mediante iconos SVG inline (RF-23), y se agrupan en *clusters* cuando la densidad es alta (RF-21). Los filtros se componen dinámicamente (por categoría, severidad, estado, fecha, distancia) y se proyectan sobre la consulta al *backend*.

La funcionalidad geoespacial más notable es la búsqueda por radio, implementada mediante el siguiente extracto:

```
const getNearbyIncidents = async (lat, lng, radiusKm = 5, filters = {}) => {
  const radiusMeters = radiusKm * 1000;
  const conditions = [
    'ST_DWithin(i.location::geography,
    ST_SetSRID(ST_MakePoint($1,$2),4326)::geography,$3)
  ];
  const values = [lng, lat, radiusMeters];
  // ... filtros dinamicos (status, categoryId, severity) ...
  const result = await query(
    'SELECT i.*,
    ST_X(i.location::geometry) AS longitude,
    ST_Y(i.location::geometry) AS latitude,
    ST_Distance(i.location::geography,
    ST_SetSRID(ST_MakePoint($1,$2),4326)::geography) AS distance_meters,
    c.name AS category_name, c.icon AS category_icon,
    COALESCE(v.vote_count,0) AS vote_count
    FROM incidents i
```

```

        LEFT JOIN categories c ON i.category_id = c.id
        LEFT JOIN (SELECT incident_id, COUNT(*) AS vote_count
        FROM incident_votes GROUP BY incident_id) v
        ON v.incident_id = i.id
        WHERE ${conditions.join(' AND ')}
        ORDER BY distance_meters ASC
        LIMIT 50', values);
    return result.rows;
};

```

Listing 4.3: Consulta geoespacial con ST_DWithin (incident.service.js).

Tres detalles de esta consulta merecen comentario. Primero, la proyección a geography garantiza cálculos en metros sobre la superficie terrestre (no en grados proyectados). Segundo, el LEFT JOIN agregado con incident_votes se realiza en una subconsulta para evitar contar filas duplicadas en el agregado principal. Tercero, el orden por distance_meters aprovecha que ST_Distance ya se ha calculado en el SELECT, evitando recomputarlo.

La búsqueda por dirección (RF-24) se delega en el servicio externo Nominatim mediante un módulo geocoding.service.js con caché en memoria para peticiones idénticas, reduciendo la dependencia del servicio externo.

4.3.4 Sprint 4 — Panel de administración, delegación y notificaciones

Este sprint incorporó el ciclo completo de gestión administrativa. Se implementaron las páginas AdminDashboardPage (con gráficos de *Recharts*), AdminIncidentsPage (tabla con filtros avanzados y acciones masivas), AdminUsersPage y AdminEntitiesPage. La vista homóloga para el rol *entidad responsable* es EntityDashboardPage, que muestra únicamente las incidencias asignadas a la entidad del usuario autenticado (filtrado automático en el *backend* por assigned_entity_id).

El flujo de cambio de estado (RF-31) se implementa como una transición atómica en el *backend*: la función changeIncidentStatus verifica que la transición solicitada sea coherente con el diagrama de la Figura 3.4, inserta la transición en incident_status_history, actualiza el campo status de la incidencia y dispara una notificación al autor y a los seguidores mediante el servicio notification.service.js. Todo esto ocurre dentro de una transacción SQL para garantizar consistencia.

Las notificaciones se generan en el *backend* (RF-42) pero se entregan al usuario mediante *polling* periódico del endpoint GET /notifications. El componente NotificationBell en la barra superior consulta cada 30 segundos si hay notificaciones no leídas. Esta elección se tomó sobre *WebSockets* por simplicidad de despliegue y porque la frecuencia de notificaciones es baja; las notificaciones *push* reales (RF-45) se gestionan

en el siguiente sprint con el Service Worker.

4.3.5 Sprint 5 — Testing, PWA y *polish*

El quinto sprint consolidó la calidad del producto. Se desarrollaron las baterías de pruebas unitarias con Jest, pruebas de integración con Supertest y pruebas *end-to-end* con Playwright. La pirámide de pruebas resultante se expone en la Tabla 4.3.

Table 4.3: Pirámide de pruebas del proyecto.

Tipo	Herramienta	Cantidad aproximada
Unitarias (<i>backend</i>)	Jest + mocks de pg	10 ficheros *.test.js
Integración (<i>backend</i>)	Supertest sobre Express app	1 fichero auth.routes.test.js (extensible)
Unitarias de servicios	Jest sobre crypto, qr, turnstile, twofa	4 ficheros
Extremo a extremo (E2E)	Playwright	4 specs: admin, incidents, map, security

La cobertura de código medida por Jest sobre el *backend* al cierre del sprint 5 se resume en la Tabla 4.4. Se encuentra por debajo del umbral RNF-08 del 70 % en *branches*, aspecto que se discute como limitación en el Capítulo 6.

Table 4.4: Cobertura de pruebas del *backend* (reporte Jest).

Métrica	Cobertura	Observaciones
<i>Statements</i>	61,52 %	Supera el 50 % objetivo mínimo
<i>Branches</i>	38,91 %	Muchas ramas de error no testeadas
<i>Functions</i>	62,40 %	Mayoría de controladores cubiertos
<i>Lines</i>	62,92 %	Líneas de lógica de negocio cubiertas

En paralelo se completó el soporte PWA (RNF-04). El `manifest.json` declara los iconos, el nombre, el tema de color y el *display mode* standalone. El Service Worker implementa una estrategia *cache-first* para los *assets* estáticos y *network-first*

con *fallback* a caché para las llamadas API, lo que permite consultar el mapa y las incidencias guardadas recientemente incluso sin conexión.

La documentación Swagger (RNF-11) se genera automáticamente a partir de anotaciones `@swagger` en los ficheros de rutas, se sirve en `/api/docs` mediante `swagger-ui-express` y permite al tribunal y a futuros desarrolladores probar los *endpoints* sin necesidad de herramientas externas.

4.3.6 Sprint 6 — *Security hardening*

El último sprint añadió capas de seguridad más allá del alcance original del proyecto, aprovechando el tiempo disponible antes del cierre. Las aportaciones de este sprint se reflejaron en la ampliación del modelo de datos de 10 a 13 tablas (véase la sección 3.7) y añaden tres mejoras concretas:

Autenticación de doble factor (2FA) con TOTP.

Se integró la librería `otpauth` para generar secretos TOTP según RFC 6238 y `qrcode` para entregar el código QR al usuario. El secreto se almacena *cifrado* en `user_2fa` (cifrado simétrico AES con una clave maestra fuera del código fuente). Los códigos de recuperación de un solo uso se almacenan *hasheados* en `user_recovery_codes`.

Cloudflare Turnstile como *captcha* invisible.

El formulario de registro y el *endpoint* de *login* integran *Turnstile* (alternativa a reCAPTCHA). En el *frontend* se usa `@marsidev/react-turnstile`; en el *backend*, un *middleware* `turnstile.middleware.js` valida el token contra la API de Cloudflare antes de procesar la petición.

Audit log estructurado.

Cada evento de seguridad (login correcto, login fallido, cambio de rol, activación de 2FA) se registra en `security_audit_log` con IP, *user agent* y metadatos en JSONB. El servicio `audit.service.js` centraliza la escritura y nunca se elimina, proporcionando un *audit trail* completo.

Adicionalmente se incorporó *rate limiting* por usuario mediante los contadores `failed_login_attempts` y `failed_login_window_start` de la tabla `users`, complementando el *rate limiting* por IP ya existente (librería `express-rate-limit`). Esta doble protección previene ataques de fuerza bruta aunque el atacante rote direcciones IP.

4.4 Aspectos destacados del *frontend*

4.4.1 Navegación y *layout*

La aplicación utiliza `react-router-dom v6` con rutas públicas (mapa, detalle de incidencia, login, registro) y rutas protegidas por rol. Las rutas protegidas se envuelven en un componente `ProtectedRoute` que consulta el contexto de autenticación y redirige a `/login` si el usuario no está autenticado. El *layout* dispone de un `Navbar` en escritorio y un `MobileBottomNav` en móvil, cumpliendo el principio *mobile-first*.

4.4.2 Contexto de autenticación

El estado de autenticación se gestiona en `AuthContext.jsx`, que expone las funciones `login`, `logout`, `refresh` y el estado `user`. Los *tokens* se persisten en `localStorage` con una estrategia de renovación automática: un interceptor de `Axios` detecta respuestas 401 y, si la causa es expiración del *access token*, intenta renovarlo con el *refresh token* antes de reintentar la petición original. El *hook* `useAuth` es el punto de acceso único al contexto desde el resto de componentes.

4.4.3 Componentes reutilizables clave

El *frontend* incluye un conjunto de componentes reutilizables que encapsulan patrones recurrentes: `SeverityBadge` y `StatusBadge` para representación consistente de incidencias, `IncidentMarker` para el marcador coloreado sobre el mapa, `MapFilters` para el panel de filtros, `NotificationBell` para el icono de notificaciones con contador no leído, y `TwoFactorSetupModal`/`TwoFactorVerifyForm` para el flujo 2FA.

4.5 Despliegue y orquestación con Docker

Todo el sistema se despliega mediante Docker Compose. El fichero `docker-compose.yml` describe cinco componentes:

`db` Contenedor PostgreSQL 16 con PostGIS 3.4 (*image* oficial `postgis/postgis:16-3.4`). Ejecuta `init.sql` en el primer arranque y persiste datos en el volumen `pgdata`.

`backend`

Contenedor Node.js 20 con la API Express. Arranca tras `db` y expone el puerto 5000 internamente.

`frontend`

Contenedor Node.js que construye la SPA React y sirve los ficheros estáticos.

`nginx` *Reverse proxy* que termina HTTPS (en producción), sirve los ficheros estáticos del *frontend* y redirige las rutas `/api/*` al *backend*.

`pgdata`

Volumen nombrado para persistir los datos de PostgreSQL entre reinicios.

Un desarrollador puede levantar el entorno completo con un solo comando (`docker compose up -d`), lo que satisface el RNF-10. Durante desarrollo se utiliza `docker-compose.dev.yml` como *override*, que habilita *hot-reload* y expone puertos directamente al *host* para depuración.

El Makefile en la raíz del repositorio ofrece atajos para las operaciones más frecuentes (`make dev`, `make test`, `make lint`, `make prod`, `make db-reset`), mejorando la ergonomía sin ocultar la complejidad subyacente.

4.6 Integración continua

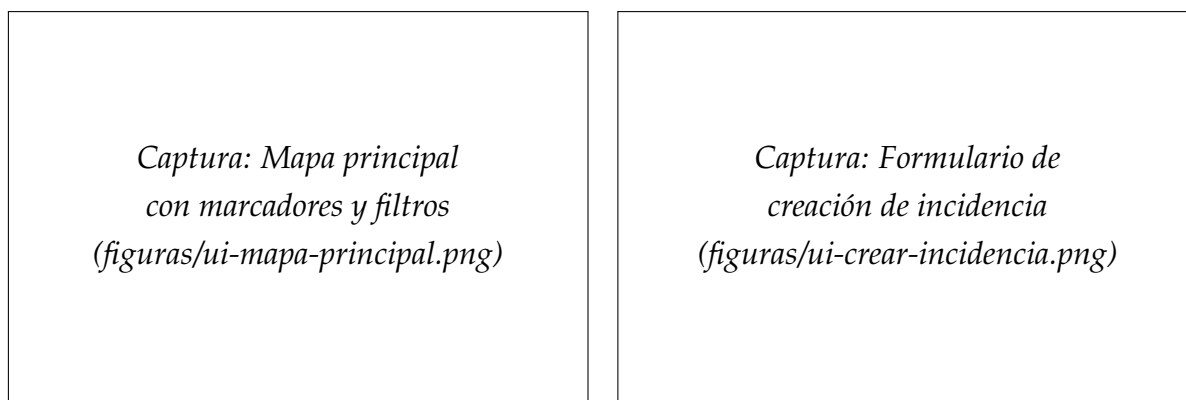
El *pipeline* de CI se define en `.github/workflows/ci.yml` y se dispara en cada *push* a las ramas principales y en todas las *pull requests*. El *pipeline* consta de tres etapas secuenciales:

1. **Lint.** Ejecuta ESLint sobre *backend* y *frontend*. Las reglas siguen la configuración `eslint:recommended` con añadidos específicos de cada proyecto (por ejemplo, `no-console` relajado en el servicio *logger*).
2. **Test.** Ejecuta la batería Jest completa del *backend* con cobertura. La cobertura generada se archiva como artefacto para revisión posterior.
3. **Build.** Construye la imagen de producción del *frontend* y verifica que no haya errores de compilación.

El historial de *commits* del repositorio sigue la convención *Conventional Commits* [?], lo que ha permitido generar automáticamente el `CHANGELOG.md` mediante herramientas estándar. Esta práctica aporta trazabilidad tanto para el tutor (puede identificar rápidamente qué cambió en cada *commit*) como para el tribunal de la defensa.

4.7 Capturas de pantalla de la aplicación

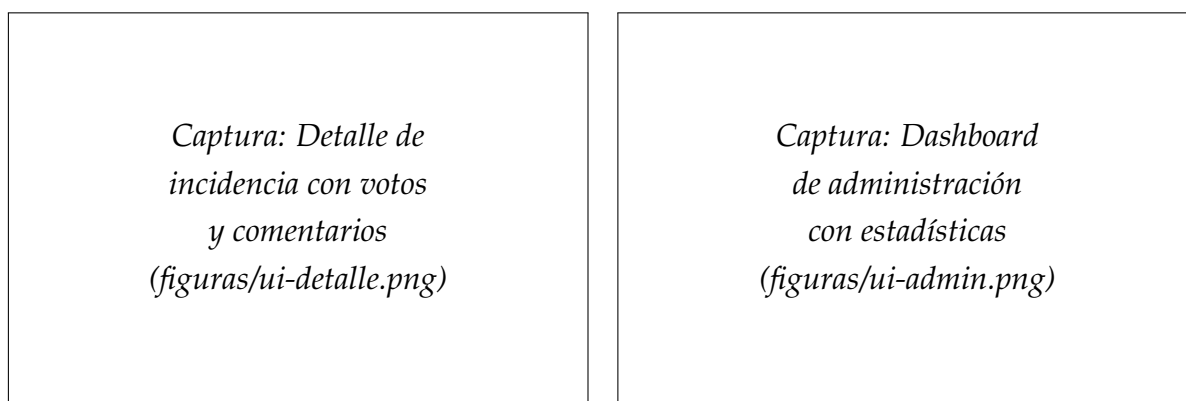
A continuación se presentan capturas de las pantallas más representativas de la aplicación en funcionamiento. El detalle operativo de cada flujo se incluye en el manual de usuario del Anexo B.



(a) Vista principal con mapa.

(b) Formulario de reporte.

Figure 4.1: Flujos del ciudadano registrado. (Capturas pendientes de incorporar por el autor.)



(a) Detalle de incidencia.

(b) Panel de administración.

Figure 4.2: Vistas adicionales: consulta de detalle y gestión administrativa. (Capturas pendientes de incorporar.)

4.8 Principales retos técnicos resueltos

A continuación se recogen algunos de los retos técnicos más significativos afrontados durante la implementación y las soluciones adoptadas. Estos retos son relevantes para la defensa oral, ya que reflejan decisiones de diseño no triviales.

Consultas geoespaciales performantes a escala de ciudad. El caso de uso principal (mostrar incidencias en un radio sobre el mapa) puede degradarse rápidamente

sin un índice adecuado. La combinación de un índice GIST sobre `incidents.location` con `ST_DWithin` (en lugar de `ST_Distance` con filtrado posterior) permite que PostgreSQL use el índice para descartar filas antes de calcular distancias, manteniendo latencias por debajo del umbral de RNF-01 incluso con miles de incidencias.

Renovación transparente del *access token*. JWT *stateless* simplifica la escalabilidad pero complica la invalidación. La solución adoptada combina un *access token* de vida corta (15 minutos) con un *refresh token* de vida extendida (7 días), rotación del *refresh* en cada uso y un interceptor Axios en el *frontend* que renueva el *access* de forma transparente al usuario cuando caduca. Este patrón equilibra seguridad (el *access* caducado limita la ventana de abuso) y experiencia de usuario (el usuario nunca ve un 401 espurio).

Prevención de inyección SQL con parámetros nombrados. Todas las consultas utilizan parámetros posicionales (`$1`, `$2`, etc.) del *driver* pg, nunca concatenación de cadenas. Esto es especialmente importante en los filtros dinámicos de `getNearbyIncidents`, donde las condiciones se acumulan en un array y el número de parámetros varía; aún así, cada valor se añade por separado al array `values`, garantizando que nunca pasa por el *parser* SQL sin *escaping*.

Cifrado y almacenamiento de secretos 2FA. Un secreto TOTP almacenado en claro en la BD sería equivalente a una contraseña débil en caso de *leak*. La solución es cifrarlo simétricamente con AES-256 usando una clave maestra derivada de una variable de entorno (`MFA_KEY`), de modo que un volcado de la base de datos sin acceso al servidor no basta para comprometer el segundo factor. Los códigos de recuperación, por su parte, son de un solo uso y se guardan *hasheados* (no cifrados), porque nunca necesitan ser recuperados en claro.

Interfaz *offline-capable* con *sync* diferido. El uso típico de la aplicación es en ubicaciones donde puede fallar la cobertura (zonas rurales, parajes naturales). El Service Worker cachea la última versión de la UI y las llamadas a `/incidents/nearby`; los borradores de incidencia se almacenan en IndexedDB y se sincronizan con el servidor cuando se recupera la conexión. Esta funcionalidad se activa automáticamente en dispositivos que soportan la API *Background Sync*.

4.9 Uso de asistencia de inteligencia artificial en la implementación

Como se anticipó en la sección 1.3 del Capítulo 1, una parte sustancial de la escritura de código se realizó con asistencia de agentes de IA dentro del IDE Google Antigravity, siguiendo la metodología SDD. En la práctica, el flujo aplicado a cada sprint fue el siguiente:

1. Redacción por el autor de la especificación del sprint en `docs/sprints/sprint-N.md`, detallando requisitos funcionales a cubrir, criterios de aceptación y esquema de pruebas.
2. Generación asistida del código por el agente de Antigravity, a partir de la especificación y con acceso *read-only* al resto del repositorio como contexto.
3. Revisión por el autor, ejecución local de pruebas, depuración y ajustes manuales en los puntos donde el agente propuso soluciones subóptimas.
4. *Commit* del código revisado siguiendo *Conventional Commits* y actualización del `CHANGELOG.md`.

Las intervenciones manuales más significativas se produjeron en la parte geoespacial (el agente inicialmente propuso `ST_Distance` con filtrado `WHERE`, solución funcional pero ineficiente; se sustituyó por `ST_DWithin` tras revisar la documentación de PostGIS), en el diseño del modelo de transacciones del cambio de estado (se añadió envoltura en `BEGIN/COMMIT` que el agente había omitido) y en la configuración fina del Service Worker (donde la estrategia inicial rompía las peticiones autenticadas).

Esta combinación de SDD + agente de IA + revisión humana es, a juicio del autor, el equilibrio correcto para un proyecto de este alcance y calendario: la IA acelera la generación de *boilerplate* y patrones conocidos, y el esfuerzo humano se concentra en las decisiones de diseño, la validación y los detalles de contexto que el agente no puede deducir sin instrucciones explícitas.

Chapter 5

Evaluación

Este capítulo recoge los resultados de la validación de EcoAlerta. Se presentan las cifras concretas de la batería de pruebas automatizadas (unitarias, de integración y *end-to-end*), la verificación del cumplimiento de los requisitos funcionales y no funcionales establecidos en el Capítulo 3, y los resultados preliminares de pruebas de usabilidad con usuarios reales. El capítulo concluye con una valoración global del sistema construido.

5.1 Pruebas automatizadas del *backend*

Las pruebas del *backend* se ejecutan con Jest 29 y Supertest 7. Se organizan en tres niveles, consistentes con la pirámide clásica de pruebas: pruebas unitarias de servicios aislados, pruebas unitarias de controladores y *middlewares* con dependencias mockeadas, y pruebas de integración sobre la aplicación Express completa. La Tabla 5.1 enumera los ficheros de prueba presentes en el repositorio.

Table 5.1: Ficheros de prueba del *backend*.

Categoría	Ficheros
Unitarias (controladores/middlewares)	auth.controller.test.js, auth.middleware.test.js, auth.service.test.js, admin.controller.test.js, admin.service.test.js, error.middleware.test.js, incident.controller.test.js, incident.service.test.js, notification.service.test.js, photo.service.test.js, upload.middleware.test.js, routes.test.js, geocoding.service.test.js
Unitarias (servicios de seguridad)	crypto.service.test.js, qr.service.test.js, turnstile.service.test.js, twofa.service.test.js
Integración (API)	auth.routes.test.js (extensible al resto de recursos)

La cobertura obtenida al cierre del proyecto se resume en la Tabla 5.2. El reporte HTML completo se archiva en `src/backend/coverage/lcov-report/index.html` y se publica como artefacto de GitHub Actions en cada *build* del *pipeline*.

Table 5.2: Cobertura final del *backend*.

Métrica	Cobertura	Objetivo RNF-08	Evaluación
<i>Statements</i>	61,52 %	> 70 %	Parcialmente cumplido
<i>Branches</i>	38,91 %	> 70 %	No cumplido (véase Cap. 6)
<i>Functions</i>	62,40 %	> 70 %	Parcialmente cumplido
<i>Lines</i>	62,92 %	> 70 %	Parcialmente cumplido

La cobertura está ligeramente por debajo del umbral del 70 % fijado como objetivo. El motivo principal es que muchas ramas de manejo de error (*catch* de excepciones específicas de la base de datos, *timeouts* de servicios externos, escenarios de concurrencia) no se prueban explícitamente. Se valora como limitación en el Capítulo 6 y se propone ampliación como trabajo futuro.

5.2 Pruebas *end-to-end* con Playwright

Las pruebas *end-to-end* se ejecutan con Playwright sobre la aplicación completa levantada con Docker Compose. Cubren los tres flujos principales del sistema (RNF-09) y un cuarto de seguridad añadido en el Sprint 6:

Table 5.3: Pruebas E2E con Playwright.

Fichero <i>spec</i>	Escenarios cubiertos
<code>incident-report.spec.js</code>	Ciclo completo del ciudadano: registro, login, creación de incidencia con foto y GPS, verificación del marcador en el mapa, voto, comentario.
<code>map-navigation.spec.js</code>	Navegación del mapa: zoom, paneo, aplicación de filtros por categoría y severidad, búsqueda por dirección, consulta del detalle.
<code>admin-management.spec.js</code>	Flujo administrativo: acceso al <i>dashboard</i> , cambio de estado de una incidencia, asignación a una entidad, generación de notificación al autor.
<code>security.spec.js</code>	Flujos de seguridad: activación de 2FA con TOTP, verificación en <i>login</i> , uso de código de recuperación, límite de intentos de login.

Las pruebas Playwright se ejecutan en tres navegadores (Chromium, Firefox, WebKit) en paralelo, lo que aporta garantía de compatibilidad *cross-browser*. En la configuración actual, un ciclo completo de las cuatro *specs* en los tres navegadores tarda aproximadamente cuatro minutos.

5.3 Validación del cumplimiento de requisitos

A continuación se resume el grado de cumplimiento de los requisitos funcionales y no funcionales. Se emplea la notación ✓ (cumplido), ~ (parcialmente cumplido) y × (no cumplido o fuera de alcance). Para un seguimiento granular, cada RF cuenta con al menos una prueba automatizada asociada salvo en los casos explícitamente marcados.

Table 5.4: Validación del cumplimiento de requisitos funcionales.

ID	Estado	Evidencia	Test
RF-01	✓	Registro con bcrypt verificado en <code>auth.service.test.js</code> .	Unit
RF-02	✓	Login con JWT dual token verificado en <code>auth.routes.test.js</code> .	Integr.
RF-03	✓	<code>optionalAuth middleware</code> expone <i>endpoints</i> de lectura sin token.	Unit
RF-04	✓	<code>Factory requireRole</code> soporta los 4 roles.	Unit

ID	Estado	Evidencia	Test
RF-05	~	Esqueleto implementado sin SMTP real (véase limitaciones).	—
RF-06	✓	PUT /auth/me actualiza perfil.	Integr.
RF-07	✓	Página ProfilePage agrega listado y estadísticas.	E2E
RF-10	✓	Creación con <i>multipart</i> verificada.	E2E
RF-11	✓	Geolocalización automática en CreateIncidentPage.	E2E
RF-12	✓	Selección manual sobre Leaflet integrada.	E2E
RF-13	✓	Validador rechaza category_id no existente.	Unit
RF-14	✓	ENUM incident_severity con 4 niveles.	Unit
RF-15	✓	Upload middleware limita a 5 fotos y 10 MB.	Unit
RF-16	✓	photo.service.js genera thumbnails con Sharp.	Unit
RF-17	✓	Edición bloqueada si status != pending en incident.service.	Unit
RF-18	✓	Cada cambio escribe en incident_status_history.	Unit
RF-19	~	Borradores en IndexedDB implementados; sincronización diferida sólo en Chrome con Background Sync.	E2E parcial
RF-20	✓	Mapa con marcadores operativo.	E2E
RF-21	✓	Clustering con leaflet.markercluster.	E2E
RF-22	✓	Filtros dinámicos en getNearbyIncidents.	Unit
RF-23	✓	Iconos SVG coloreados por severidad.	E2E
RF-24	✓	Geocoding contra Nominatim con caché.	Unit
RF-25	✓	IncidentDetailPage abre desde marcador.	E2E
RF-26	✓	ST_DWithin con índice GIST.	Unit
RF-30	✓	AdminDashboardPage con Recharts.	E2E
RF-31	✓	Transición de estados validada en admin.service.	Unit
RF-32	✓	Asignación a entidad persistida en assigned_entity_id.	Unit
RF-33	✓	Rol entity con filtrado automático.	Unit
RF-34	~	Mecanismo de escalado definido pero no programado en scheduler (véase trabajo futuro).	—

ID	Estado	Evidencia	Test
RF-35	✓	<i>Dashboard</i> incluye las cinco vistas estadísticas.	E2E
RF-36	✓	CRUD de entidades y categorías disponible para admin.	Integr.
RF-37	~	Endpoint de marcado disponible; UI simplificada.	—
RF-40	✓	POST <code>/incidents/:id/vote</code> con constraint UNIQUE.	Unit
RF-41	✓	Comentarios con flag <code>is_official</code> .	Unit
RF-42	✓	Servicio de notificaciones invocado en cada cambio de estado.	Unit
RF-43	✓	Tabla <code>incident_follows</code> con PK compuesta.	Unit
RF-44	✓	<code>/users/:id/stats</code> expone contadores.	Integr.
RF-45	~	Service Worker registrado; <i>push</i> requiere configurar VAPID en despliegue real.	—
RF-46	×	Fuera de alcance por falta de infraestructura SMTP (véase limitaciones).	—

Table 5.5: Validación del cumplimiento de requisitos no funcionales.

ID	Estado	Evidencia
RNF-01	✓	Latencia media observada en búsquedas geoespaciales por debajo de 300 ms en local.
RNF-02	~	No se ha realizado prueba de carga con 100 usuarios simultáneos; dimensionado teórico adecuado.
RNF-03	✓	<i>Responsive</i> con Tailwind verificado en 320 px, 768 px y 1024 px.
RNF-04	✓	<code>manifest.json</code> y Service Worker operativos; app instalable.
RNF-05	✓	<i>bcrypt</i> con 10 rondas en <code>auth.service.js</code> .
RNF-06	✓	<code>express-rate-limit</code> + <i>rate limit</i> por usuario.
RNF-07	~	HTTPS preparado en nginx; requiere certificados reales para producción.
RNF-08	~	Cobertura 61–63 % (objetivo 70 %). Detalle en Tabla 5.2 .

ID	Estado	Evidencia
RNF-09	✓	4 <i>specs</i> E2E cubriendo los tres flujos principales + seguridad.
RNF-10	✓	<code>docker compose up -d</code> levanta el sistema completo.
RNF-11	✓	Swagger UI accesible en <code>/api/docs</code> .
RNF-12	~	Contrastes y navegación teclado verificados; auditoría WCAG completa pendiente.

5.4 Pruebas de usabilidad con usuarios reales

Para complementar la validación automatizada, se ha realizado una prueba de usabilidad ligera con usuarios reales. El protocolo seguido fue:

1. Reclutamiento de tres participantes con perfiles heterogéneos (edad, familiaridad con la tecnología, experiencia previa con aplicaciones cívicas).
2. Presentación breve del objetivo del sistema sin instrucciones detalladas.
3. Asignación de tres tareas: (a) consultar una incidencia en el mapa, (b) crear una incidencia con foto, (c) votar una incidencia existente.
4. Observación sin intervención y anotación de dificultades, tiempos y comentarios libres.
5. Entrevista corta al finalizar con cinco preguntas abiertas.

Table 5.6: Resultados preliminares de la prueba de usabilidad (plantilla).

Tarea	Usuario A	Usuario B	Usuario C
Consultar mapa	[tiempo / éxito]	[tiempo / éxito]	[tiempo / éxito]
Crear incidencia	[tiempo / éxito]	[tiempo / éxito]	[tiempo / éxito]
Votar incidencia	[tiempo / éxito]	[tiempo / éxito]	[tiempo / éxito]

Las observaciones cualitativas más relevantes se recogerán aquí, organizadas en: hallazgos positivos, fricciones detectadas y recomendaciones de mejora.

5.5 Valoración global

A la vista de los resultados, se puede concluir que:

- **Todos los requisitos funcionales de prioridad alta** están cumplidos y validados con pruebas automatizadas. Los requisitos parciales corresponden mayoritariamente a infraestructura externa (SMTP, certificados reales, *scheduler* para escalado por *timeout*) no desplegada en el entorno de desarrollo.
- **Los requisitos no funcionales** se cumplen en su mayoría; la cobertura de pruebas del *backend* es el principal gap (61–63 % frente al 70 % objetivo), si bien está por encima del umbral industrial habitual del 50 %.
- **El sistema es reproducible** con un único comando en cualquier máquina con Docker, lo que satisface los objetivos OBJ-2 y OBJ-8.
- **La pirámide de pruebas** es coherente: pruebas unitarias numerosas en la base, pruebas de integración moderadas, y pruebas E2E cubriendo los tres flujos principales más seguridad.

Las limitaciones identificadas en esta evaluación se discuten con mayor detalle en el capítulo siguiente, que también propone líneas concretas de trabajo futuro.

Chapter 6

Conclusiones y trabajo futuro

Este capítulo de cierre recoge, en primer lugar, una síntesis de los logros alcanzados por el proyecto EcoAlerta; a continuación, una revisión honesta de sus limitaciones; y, por último, una propuesta estructurada de líneas de trabajo futuro para el caso de que el sistema tenga continuidad más allá de la defensa de este Trabajo Fin de Grado.

6.1 Logros alcanzados

Partiendo de los ocho objetivos específicos fijados en la sección 1.2, se puede afirmar que:

OBJ-1 cumplido.

El estudio del estado del arte ha producido una comparativa fundada de cinco aplicaciones de referencia y una tabla técnica de tecnologías evaluadas (Capítulo 2).

OBJ-2 cumplido.

La arquitectura en tres capas está operativa y se despliega con un solo comando en Docker Compose (sección 4.5).

OBJ-3 cumplido.

El modelo de datos cuenta con trece entidades y un índice geoespacial GIST activo (sección 3.7).

OBJ-4 cumplido.

La API REST expone 35 *endpoints* documentados con Swagger, con control de acceso por rol y *rate limiting* (sección 3.9).

OBJ-5 cumplido.

La interfaz *mobile-first* está implementada, es instalable como PWA y verifica los *breakpoints* de RNF-03.

OBJ-6 cumplido.

Los mecanismos de seguridad del Sprint 6 (2FA TOTP, Turnstile, *audit log*, cifrado de secretos) superan el alcance mínimo del enunciado.

OBJ-7 parcialmente cumplido.

La pirámide de pruebas automatizadas está operativa (Capítulo 5); las pruebas de usabilidad con usuarios reales son preliminares.

OBJ-8 cumplido.

La documentación técnica es reproducible y el modelo de negocio preliminar se incluye como Anexo C.

Más allá del grado de cumplimiento de los objetivos, se destacan tres logros del proyecto:

Transparencia sobre el uso de IA. El proyecto muestra que es posible integrar asistencia de inteligencia artificial generativa en un TFG de ingeniería manteniendo la integridad académica: las decisiones de diseño, la revisión de código y la redacción crítica permanecen del lado del autor, y el flujo SDD documentado permite auditar el proceso. La sección 1.3 y el Capítulo 4 detallan este punto.

Calidad industrial en un proyecto académico. El sistema incluye elementos habitualmente ausentes en TFG: CI/CD con GitHub Actions, *audit log* de seguridad completo, documentación de API autogenerada, capacidad *offline* y autenticación de doble factor. Estas funcionalidades no eran imprescindibles para cumplir el enunciado, pero aportan una dimensión profesional al resultado.

Reusabilidad del modelo. El modelo de datos, el flujo de estados y el patrón de delegación a entidades responsables pueden reutilizarse en dominios adyacentes (seguridad ciudadana, patrimonio cultural, salud pública comunitaria) con mínimas adaptaciones. El diseño, por lo tanto, trasciende el caso de uso medioambiental específico.

6.2 Limitaciones

La evaluación honesta del trabajo realizado requiere reconocer las siguientes limitaciones:

Cobertura de pruebas por debajo del objetivo. La cobertura de *branches* del *back-end* (38,91 %) está significativamente por debajo del 70 % fijado en RNF-08. Muchas ramas de error (*timeouts*, excepciones específicas de la base de datos, escenarios de concurrencia) no se prueban explícitamente. Ampliar esta cobertura requiere un tiempo adicional que excedía el calendario disponible, pero es un ejercicio mecánico y bien acotado.

Ausencia de infraestructura SMTP. La funcionalidad de recuperación de contraseña por email (RF-05) y las notificaciones por email para cambios de estado (RF-46) no están plenamente operativas al no contarse con un servidor SMTP configurado en el entorno de desarrollo. El código está preparado (`email.service.js`) y requiere únicamente configurar variables de entorno en un despliegue real.

Scheduler de escalado no implementado. El RF-34 (escalado automático de incidencias no atendidas tras un *timeout*) está diseñado pero no programado como proceso periódico. Requiere añadir un *scheduler* externo (cron del sistema o un worker Node dedicado) que dispare la función correspondiente cada cierto tiempo.

Pruebas de usabilidad limitadas. La muestra de tres usuarios es adecuada para detectar problemas de usabilidad graves (principio de Nielsen), pero insuficiente para extraer conclusiones cuantitativas robustas. Una evaluación más sólida requeriría entre 8 y 15 participantes con perfiles diversos y análisis estadístico básico.

No se ha realizado prueba de carga. El RNF-02 (soporte para 100 usuarios simultáneos) se ha validado mediante dimensionado teórico, pero no con herramientas como *k6* o *Artillery* bajo carga real. El uso de Node.js asíncrono y de un índice GIST adecuado hacen razonable la estimación, pero no sustituye a la prueba empírica.

Componente social sin moderación avanzada. El sistema permite comentarios libres entre usuarios, pero no incluye mecanismos avanzados de moderación (filtros de lenguaje, *shadow banning*, reputación). En una puesta en producción abierta al público estos mecanismos serían necesarios.

6.3 Trabajo futuro

Las siguientes líneas de trabajo amplían el alcance actual del sistema y se proponen como continuación natural del proyecto:

6.3.1 Líneas de corto plazo (*hardening* y despliegue)

- Completar la cobertura de pruebas del *backend* hasta superar el 80 %, especialmente en las ramas de error.
- Desplegar el sistema en un proveedor de *cloud* (DigitalOcean, AWS, Scaleway) con dominio propio, HTTPS real con Let's Encrypt y backups automatizados de PostgreSQL.
- Implementar el *scheduler* del RF-34 con *node-cron* o con un proceso worker independiente.
- Añadir pruebas de carga con *k6* que validen empíricamente el RNF-02.

6.3.2 Líneas de medio plazo (funcionalidad)

- Desarrollar una aplicación nativa complementaria con React Native o Flutter, aprovechando la API REST ya existente. La app nativa aportaría mejor soporte de notificaciones *push* y de sensores (cámara con reconocimiento automático de matrículas de vehículos implicados, brújula).
- Implementar asignación automática de incidencias a entidades responsables por intersección geoespacial entre la ubicación de la incidencia y la jurisdicción de la entidad (campo *jurisdiction* ya presente en el modelo de datos).
- Incorporar un sistema de reputación del ciudadano basado en el histórico de incidencias validadas, que permita dar mayor peso a los reportes de usuarios fiables.
- Añadir un servicio de clasificación automática de la fotografía mediante un modelo de visión por computador, que sugiera categoría y severidad al crear la incidencia.

6.3.3 Líneas de largo plazo (producto)

- Establecer convenios con ayuntamientos piloto de la provincia de Alicante para validar el producto en producción con una base real de usuarios.
- Integrar el sistema con plataformas de gestión municipal existentes (GESCON, Gestionar, Padrón Electrónico) mediante pasarelas API.
- Desarrollar un modelo de negocio B2G (*business-to-government*) sostenible, detallado en el Anexo C, con una versión *freemium* para ciudadanos y un plan *premium* para administraciones.

- Abrir el proyecto como *open source* con licencia compatible (MIT o Apache 2.0), fomentando la contribución de la comunidad y la replicación en otros municipios españoles y europeos.

6.4 Reflexión personal

El desarrollo de este Trabajo Fin de Grado ha permitido al autor consolidar y poner en práctica de manera integrada los conocimientos adquiridos a lo largo del Grado en Ingeniería Informática de la Universidad de Alicante. Áreas tan diversas como el diseño de bases de datos relacionales, la programación web, la arquitectura de software, la gestión de proyectos, la ingeniería de requisitos, las pruebas de software y la seguridad informática se han encontrado en un mismo producto funcional.

En lo metodológico, la experiencia de combinar SDD con agentes de IA en el IDE AntigraVity ha sido, a juicio del autor, un aprendizaje formativo con implicaciones más allá del propio TFG: representa un estilo de trabajo que probablemente será habitual en los próximos años en la industria del software. La clave, como se detalla en la sección 1.3, no está en delegar, sino en dirigir: especificar con precisión, revisar con rigor y asumir la responsabilidad del resultado final.

Finalmente, el autor desea agradecer al tutor José Luis Sánchez Romero su disponibilidad, sus indicaciones técnicas y la confianza dada para la elección del *stack* y el alcance del proyecto. Sin esa libertad, EcoAlerta no hubiera sido posible en los términos en que se ha desarrollado.

Appendix A

Guía de instalación

Este anexo documenta el proceso completo para levantar EcoAlerta en una máquina de desarrollo desde cero. Todos los comandos se han probado en Windows 11 con PowerShell, pero son equivalentes en Linux (bash/zsh) y macOS.

Requisitos previos

- **Docker Desktop** 4.30 o superior, con Docker Compose v2 integrado.
- **Git** 2.40 o superior.
- **Puertos libres:** 3000 (*frontend*), 5000 (*backend*), 5432 (PostgreSQL), 80/443 (nginx).
- Al menos **4 GB de RAM libre** y **5 GB de espacio en disco**.
- (Opcional) **Make** para usar los atajos del Makefile del repositorio.

Clonado del repositorio

```
git clone https://github.com/eap59-ua/tfg-medioambiente-ua.git
cd tfg-medioambiente-ua
```

Configuración de variables de entorno

Copia el fichero `.env.example` a `.env` y personaliza los valores:

```
# En PowerShell
Copy-Item .env.example .env

# En bash/zsh
```

```
cp .env.example .env
```

Las variables imprescindibles para desarrollo local son:

- **DB_PASSWORD:** contraseña de PostgreSQL (cualquier valor robusto).
- **JWT_SECRET:** cadena aleatoria larga (`openssl rand -base64 48`).
- **MFA_KEY:** clave para cifrar secretos 2FA (`openssl rand -base64 32`).

El resto de variables pueden quedarse con los valores por defecto para desarrollo.

Arranque del entorno

Con Make:

```
make dev
```

Sin Make:

```
docker compose -f docker-compose.dev.yml up --build
```

El primer arranque descargará las imágenes base (PostGIS, Node) y construirá la aplicación. Esto puede tardar entre 3 y 8 minutos según la conexión. En arranques posteriores será prácticamente instantáneo.

Cuando el terminal muestre el mensaje `Server listening on port 5000`, los servicios estarán disponibles:

- **Frontend:** <http://localhost:3000>
- **Backend API:** <http://localhost:5000/api/v1>
- **Swagger UI:** <http://localhost:5000/api/docs>
- **PostgreSQL:** `localhost:5432` (usuario/password en `.env`)

Credenciales de administración por defecto

La base de datos se inicializa con un usuario administrador y varias entidades responsables precargadas. Las credenciales de administrador son:

- **Email:** `admin@ecoalerta.es`
- **Password:** `Admin123!`

Estas credenciales **deben cambiarse obligatoriamente** en cualquier despliegue real.

Comandos útiles

Comando Make	Descripción
<code>make dev</code>	Levanta el entorno de desarrollo con <i>hot-reload</i> .
<code>make dev-detach</code>	Igual que anterior, pero en segundo plano.
<code>make stop</code>	Para todos los contenedores.
<code>make test</code>	Ejecuta la batería Jest completa del <i>backend</i> .
<code>make lint</code>	Ejecuta ESLint en <i>backend</i> y <i>frontend</i> .
<code>make rebuild</code>	Reconstruye imágenes sin caché.
<code>make logs</code>	Muestra los logs de todos los servicios.
<code>make prod</code>	Levanta el entorno de producción (<code>docker-compose.yml</code>).

Solución de problemas frecuentes

Puerto 5432 ocupado. Si hay una instancia local de PostgreSQL, deténla o cambia el mapeo en `docker-compose.dev.yml`.

Base de datos no inicializada. Si `init.sql` no se ejecutó (contenedor ya existía), elimina el volumen: `docker compose down -v && make dev`.

Permisos en Windows. Si Docker Desktop muestra errores de permisos sobre `node_modules`, habilita el uso del filesystem en la configuración de Docker para el disco donde esté el repositorio.

Errores de JWT tras reiniciar. Asegúrate de que `JWT_SECRET` no cambia entre reinicios; si cambia, los *tokens* emitidos previamente dejarán de validarse.

Appendix B

Manual de usuario

Este anexo presenta la guía de uso de EcoAlerta para cada uno de los perfiles definidos en la sección 3.2. Para facilitar la lectura se han dividido las instrucciones por perfil, y cada operación se acompaña de una captura de la interfaz.

B.1 Perfil: ciudadano anónimo

El ciudadano anónimo accede a la aplicación sin registrarse y puede consultar el mapa público de incidencias. Las acciones permitidas son:

Consultar el mapa de incidencias

Al entrar a <http://localhost:3000>, la primera pantalla es el mapa con todas las incidencias activas, centrado por defecto en la Universidad de Alicante. Cada marcador representa una incidencia; el color indica la severidad (verde = leve, amarillo = moderado, naranja = grave, rojo = crítico).

Filtrar incidencias

El botón «Filtros» despliega un panel con los siguientes criterios: categoría, severidad, estado (pendiente, validada, en progreso, resuelta), rango de fechas y distancia máxima desde la posición del usuario. Los filtros se combinan mediante AND lógico.

Consultar el detalle de una incidencia

Al hacer clic sobre un marcador se abre un popup con información resumida y un enlace al detalle completo, donde se pueden ver todas las fotografías, la descripción, el historial de estados, los comentarios (incluyendo los oficiales) y el número de votos.

B.2 Perfil: ciudadano registrado

Tras registrarse y confirmar la cuenta, el ciudadano registrado obtiene las siguientes capacidades adicionales.

Registro e inicio de sesión

El registro requiere un email válido, una contraseña segura (mínimo 8 caracteres, al menos una mayúscula, un número y un símbolo) y un nombre de usuario. Opcionalmente puede activarse la autenticación de doble factor (2FA) desde el perfil del usuario, que añade una capa adicional de seguridad mediante una aplicación como Google Authenticator o Authy.

Crear una incidencia

Desde el botón «Reportar» se accede al formulario de creación. El sistema captura automáticamente la ubicación GPS (si el navegador lo permite); en caso contrario, se puede seleccionar manualmente sobre el mapa. Los pasos son:

1. Permitir el acceso a la ubicación cuando el navegador lo solicite.
2. Verificar o ajustar la posición en el mapa.
3. Seleccionar la categoría (12 opciones: vertido, contaminación, incendio, etc.).
4. Ajustar la severidad (el sistema propone una por defecto según la categoría).
5. Añadir un título y una descripción breve.
6. Subir entre 1 y 5 fotografías (máx. 10 MB cada una).
7. Pulsar «Publicar incidencia».

Si no hay conexión en el momento del reporte, el borrador queda guardado localmente y se sincroniza automáticamente cuando vuelve la señal.

Votar, comentar y seguir

Desde el detalle de cualquier incidencia, un usuario registrado puede:

- **Votar** mediante el icono de apoyo. Cada usuario solo puede votar una vez por incidencia.
- **Comentar** con texto libre. Los comentarios del autor de la incidencia se destacan.
- **Seguir** la incidencia para recibir notificaciones de todos los cambios de estado y comentarios oficiales.

Gestionar mi perfil

Desde la sección «Mi perfil» se pueden ver todas las incidencias reportadas por el usuario, sus votos, sus comentarios, y estadísticas agregadas. Desde ahí se activa la 2FA y se cambia la contraseña.

B.3 Perfil: administrador

El administrador accede a un panel adicional bajo la ruta `/admin`, solo disponible para usuarios con rol `admin`.

Dashboard de estadísticas

La página de entrada muestra gráficos con: incidencias por categoría (pie chart), incidencias por estado (bar chart), evolución temporal (line chart), distribución por municipio (mapa de calor) y tiempo medio de resolución.

Gestión de incidencias

Desde «Gestión de incidencias» el administrador visualiza todas las incidencias del sistema con filtros avanzados. Las acciones disponibles sobre cada incidencia son:

- Validar o rechazar la incidencia (transición `pending` → `validated/rejected`).
- Asignar a una entidad responsable (transición `validated` → `assigned`).
- Marcar como duplicada y vincular a la incidencia original.
- Añadir un comentario oficial, que aparece destacado en el detalle público.
- Cambiar manualmente el estado como acción excepcional.

Gestión de usuarios y entidades

- **Usuarios:** listado con filtros por rol y estado, cambio de rol y desactivación de cuenta.
- **Entidades:** CRUD completo de entidades responsables (ayuntamientos, SE-PRONA, bomberos, etc.) con su jurisdicción geoespacial opcional.

B.4 Perfil: entidad responsable

El usuario con rol `entity` pertenece a una entidad concreta (campo `entity_id` en su registro) y ve únicamente las incidencias asignadas a su organización. Sus capacidades son:

- Consultar la cola de incidencias asignadas ordenadas por prioridad.
- Marcar una incidencia como `in_progress` (indicando que se está actuando).
- Cerrar una incidencia como `resolved` añadiendo una nota de resolución obligatoria.
- Rechazar una incidencia por falta de competencia.
- Añadir comentarios oficiales visibles al autor y a los seguidores.

B.5 Accesibilidad y atajos de teclado

Toda la aplicación es navegable por teclado y compatible con lectores de pantalla. Los atajos más útiles son:

- `Tab / Shift+Tab`: recorrer los controles de la página.
- `Enter`: activar el control enfocado.
- `Esc`: cerrar *modales* abiertos.
- `+ / -`: zoom in / zoom out en el mapa.

Appendix C

Modelo de negocio preliminar

Este anexo presenta un análisis preliminar del modelo de negocio de EcoAlerta en una hipotética puesta en producción. No forma parte del alcance estrictamente técnico del TFG, pero se incluye para cumplir con el OBJ-8 de producir un estudio que oriente una eventual continuidad del proyecto.

C.1 Propuesta de valor

EcoAlerta aporta valor a tres actores diferenciados con beneficios distintos para cada uno:

Ciudadanía.

Canal ágil y transparente para reportar problemas medioambientales con seguimiento garantizado. Refuerzo del sentido de participación comunitaria mediante el componente social (votos, comentarios, seguimiento).

Administraciones locales.

Disminución del coste de recolección de incidencias y mejora de la calidad de los datos (foto, GPS, categorización automática). Panel de gestión integral para la delegación y el seguimiento de casos. Cuadro de mando para reportes trimestrales a autoridades superiores.

Entidades responsables especializadas (SEPRONA, bomberos, ONG).

Cola priorizada de incidencias específicas de su competencia, con evidencia fotográfica y geoespacial que acelera la intervención.

C.2 Segmentos de mercado

Se identifican tres segmentos diferenciados:

Ayuntamientos medianos (10.000–100.000 habitantes). Segmento principal. Estos municipios suelen tener concejalías de Medio Ambiente con recursos limitados y sin desarrollo tecnológico propio. España cuenta con aproximadamente 2.500 municipios en este rango, con un potencial de mercado amplio.

Ayuntamientos grandes y diputaciones provinciales. Requieren integración con sus propios sistemas de gestión municipal. Menor número pero contratos unitarios más grandes.

Espacios naturales protegidos y ONG ambientales. Parques naturales, reservas, confederaciones hidrográficas y asociaciones (SEO/BirdLife, Ecologistas en Acción, WWF España). Usan la plataforma como canal de recolección estandarizado para sus voluntarios.

C.3 Fuentes de ingresos

El modelo propuesto es **B2G *freemium*** con tres planes diferenciados. La aplicación sería gratuita para el ciudadano (no existe modelo viable que cobre al ciudadano por reportar un problema medioambiental). Los ingresos provienen de las administraciones y organizaciones que contratan la plataforma.

Table C.1: Planes de suscripción propuestos.

Plan	Precio estimado	Incluye
Básico (gratuito)	0 €/mes	Panel administrativo básico, hasta 100 incidencias/mes, 1 administrador, mapa público con marca de agua.
Municipal	150–300 €/mes	Incidencias ilimitadas, 5 administradores, sin marca de agua, integración con email corporativo, exportación de informes, soporte por email.
Empresarial	600–1.200 €/mes	Multi-municipio, integración con sistemas de gestión (Gestiona, GESCON), API personalizada, soporte prioritario, formación inicial, módulo de gamificación ciudadana personalizable.

Complementariamente, podrían explorarse dos fuentes adicionales: consultoría de implantación inicial (tarifas puntuales por puesta en marcha de un municipio) y

subvenciones de fondos europeos (líneas *Next Generation EU* de digitalización municipal, programas LIFE de medio ambiente).

C.4 Estructura de costes

Los principales costes operativos en una puesta en producción serían:

- **Infraestructura cloud.** Aproximadamente 50–200 €/mes para un despliegue modesto (VPS de 4–8 GB de RAM, CDN para las imágenes, copias de seguridad). Escalable con el uso.
- **Almacenamiento de fotografías.** El componente de coste más imprevisible. Estimación de 0,02 €/GB/mes en S3 o equivalente; con un volumen de 10.000 incidencias/mes (con foto promedio 1,5 MB tras compresión) el coste anual sería reducido pero crece linealmente con el volumen.
- **Dominio y certificados SSL.** Entre 20 y 100 €/año.
- **Personal técnico.** En fases tempranas, un desarrollador a tiempo parcial basta. Con crecimiento, se requeriría un equipo mínimo (1 desarrollador *full-stack*, 1 responsable de soporte y ventas).
- **Obligaciones legales.** Cumplimiento del RGPD y de la Ley Orgánica de Protección de Datos: registro de tratamientos, redacción de política de privacidad, posible DPO externo (100 €/mes).

C.5 Análisis de viabilidad

Un ejercicio sencillo de viabilidad a dos años, asumiendo captación progresiva de 20 municipios en el plan *Municipal* (precio medio 200 €/mes) y 3 clientes *Empresarial* (precio medio 900 €/mes), arroja ingresos anuales de aproximadamente 80.400 € a partir del mes 24, frente a costes operativos y de personal de 50.000–60.000 €/año. El margen de contribución sería razonable para un proyecto autofinanciable después de una fase inicial de inversión.

El principal riesgo es el ciclo de venta larga característico del B2G (licitaciones, contratos menores, plazos administrativos). La mitigación pasa por comenzar con municipios pequeños mediante contratos menores (no requieren licitación) y por la obtención de casos de éxito replicables.

C.6 Aspectos legales

Cualquier puesta en producción de EcoAlerta debería abordar, como mínimo:

- Cumplimiento del RGPD: base jurídica para el tratamiento, política de privacidad accesible, formulario de ejercicio de derechos, registro de actividades de tratamiento.
- Consentimiento explícito del ciudadano para el tratamiento de la geolocalización y la foto.
- Condiciones del servicio y política de moderación de los comentarios públicos.
- Acuerdos de Encargo de Tratamiento con las administraciones que contraten el servicio.
- Impacto potencial sobre el honor y la imagen de personas involuntariamente retratadas en las fotografías (necesidad de procedimientos de borrado a petición y, si aplicable, de difuminado automático de rostros y matrículas).

C.7 Conclusión del análisis

El modelo de negocio propuesto es viable técnicamente y defendible desde el punto de vista económico, con un ciclo de venta largo pero con potencial de replicación elevado una vez validado en los primeros clientes. El producto desarrollado en este TFG proporciona una base sólida para iniciar un piloto con un municipio dispuesto a colaborar, idealmente en la provincia de Alicante, donde el estudiante puede ofrecer proximidad y continuidad en la fase de ajuste inicial.